



Data structures and its applications

Unit - II

Compiled by
M S Anand

Department of Computer Science (AI&ML)

Data Structures and its applications

Introduction



Text Book(s):

1. "Data Structures using C / C++", Langsum Yedidiah, Moshe J Augenstein, Aaron M Tenenbaum Pearson Education Inc, 2nd edition, 2015.

Reference Book(s):

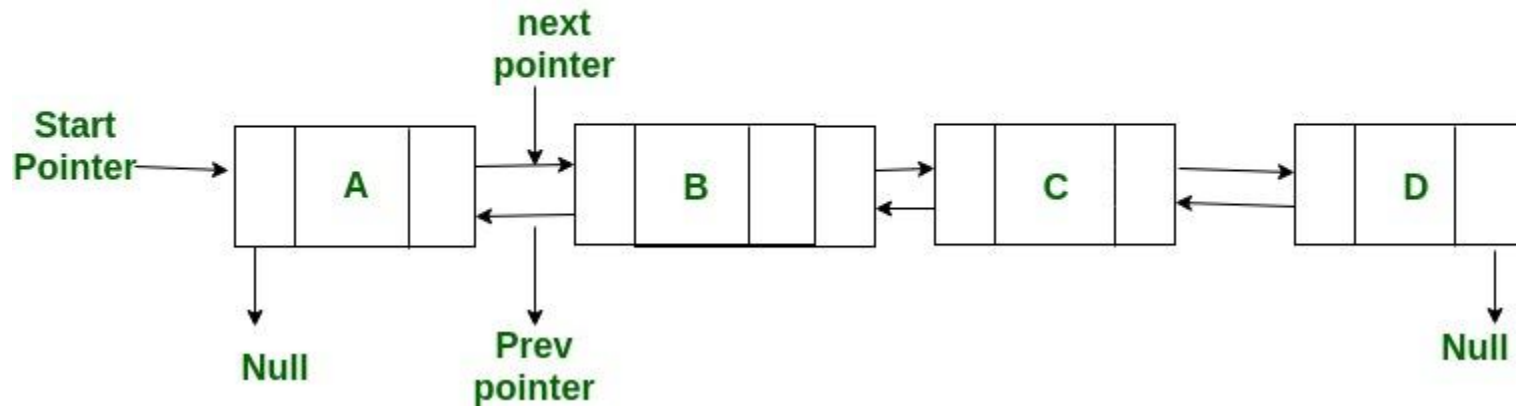
1. "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019

Data structures and its applications

Linked list

Doubly Linked List

In computer science, a **doubly linked list** is a **linked** data structure that consists of a set of sequentially **linked** records called nodes. Each node contains three fields: two **link** fields (references to the previous and to the next node in the sequence of nodes) and one data field.



Data structures and its applications

Linked list

How do you represent a doubly linked list in C?

/* Node of a doubly linked list */

struct Node

```
{  
    int data;  
    struct Node* next; // Pointer to next node in DLL  
    struct Node* prev; // Pointer to previous node in DLL  
};
```



Advantages of DLL over SLL

1. A DLL can be traversed in both forward and backward direction.
2. The delete operation in DLL is more efficient if pointer to the node to be deleted is given.
3. We can quickly insert a new node before a given node.
In singly linked list, to delete a node, pointer to the previous node is needed. To get this previous node, sometimes the entire list is traversed. In DLL, we can get the “previous” node using “previous” pointer.

Disadvantages over SLL

1. Every node of DLL requires extra space for a “previous” pointer.
2. All operations require an extra pointer “previous” to be maintained. For example, in insertion, we need to modify previous pointers together with next pointers.

Data structures and its applications

Linked list

Operations on doubly linked list

Creation

Insertion of a node at a given position

Deletion from a given position

Traversal

Sample implementation:

[MainProgDLL_2.c](#)

[DLList_2.c](#)

[DLList_2.h](#)

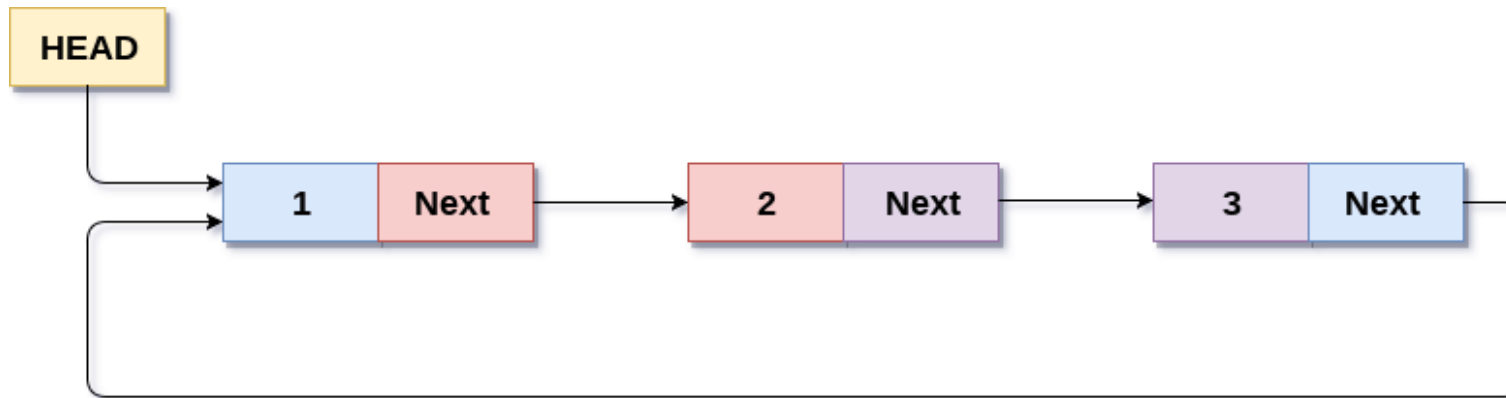


Data structures and its applications

Linked list

Circularly linked list

A **circular linked list** is a **linked list** in which the last node points to the head or first node making the data structure to look like a circle. A **circularly linked list** node can be implemented using singly linked or doubly **linked list**.



Circular Singly Linked List

Sample implementation: [MainCircularSLL.c](#) [CircularList.h](#) [CircularSLList.c](#)

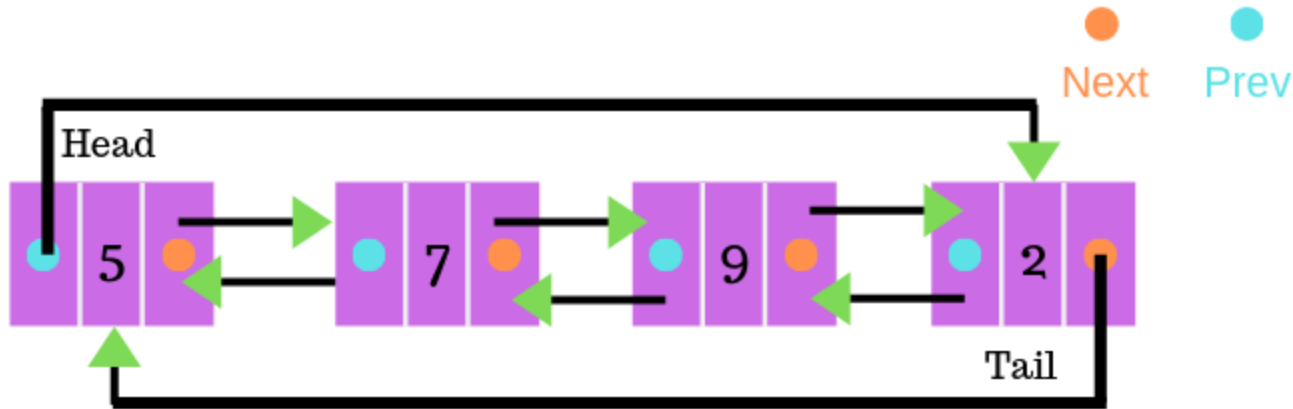
[MainCircularSLL2.c](#) [CircularList2.h](#) [CircularSLList2.c](#)

Data structures and its applications

Linked list

Circularly linked list

Implementation using doubly linked list:



[MainProgCircularDLL.c](#)

[CircularDLLList.c](#)

[CircularDLLList.h](#)

Advantages of Circular Linked Lists:

1. Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.
2. Useful for implementation of queue. We don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.
3. Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application.

Queues

A Queue is a linear structure which follows a particular order in which the operations are performed.

The order is First In First Out (FIFO).

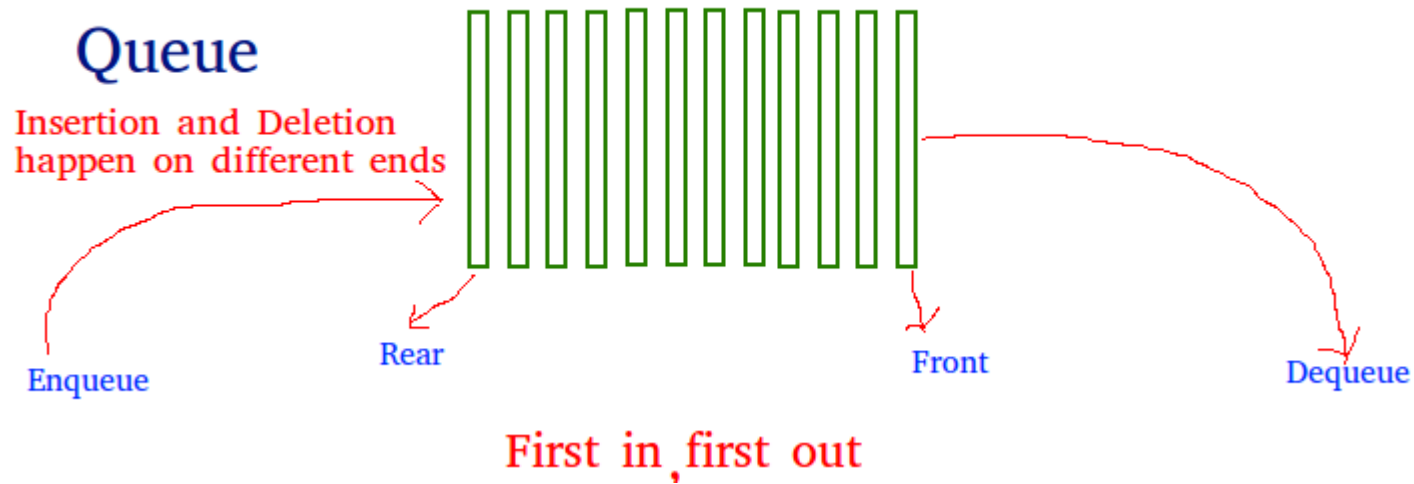
A good example of a queue is any queue of consumers for a resource where the consumer who came first is served first.

The difference between stacks and queues is in removing.

In a stack we remove the item the most recently added; in a queue, we remove the item the least recently added.

Data structures and its applications

Queue



Data structures and its applications

Queue

Basic operations on a queue

Enqueue() - Insertion of elements to the queue at the rear end.

Dequeue() - Removal of elements from the queue (from the head).

Peek() - Acquires the data element available at the front node of the queue without deleting it.

isFull() - Checks if the queue is full.

isNull() - Checks if the queue is empty.



Data structures and its applications

Queue

Enqueue() Operation

The following steps should be followed to insert (enqueue) data element into a queue -

Step 1: Check if the queue is full.

Step 2: If the queue is full, Overflow error.

Step 3: If the queue is not full, increment the rear pointer to point to the next available empty space.

Step 4: Add the data element to the queue location where the rear is pointing.



Dequeue operation

Step 1: Check if the queue is empty.

Step 2: If the queue is empty, Underflow error.

Step 3: If the queue is not empty, access the data where the front pointer is pointing.

Step 4: Increment front pointer to point to the next available data element.

Data structures and its applications

Queue

Peek () operation

.

Step 1: Check if the queue is empty.

Step 2: If the queue is empty, print “Queue is Empty.”

Step 3: If the queue is not empty, access the data where the front pointer is pointing.

Step 4: Return data.



Data structures and its applications

Queue



isFull() Operation

This function checks if the queue has the maximum allowed number of elements.

The following steps are performed in the isFull() operation -

Step 1: Check if $\text{rear} == \text{MAXSIZE} - 1$.

Step 2: If they are equal, return "Queue is Full."

Step 3: If they are not equal, return "Queue is not Full."

Data structures and its applications

Queue



isEmpty() Operation

.

The algorithm of the isEmpty() operation is as follows -

Step 1: Check if the rear and front are pointing to null memory space.

Step 2: If they are pointing to NULL, return “Queue is empty.”

Step 3: If they are not equal, return “Queue is not empty.”

Data structures and its applications

Queue



Implementation using arrays

[QueuewithAMain.c](#) [QueuewithArrays.c](#) [QueueArray.h](#)

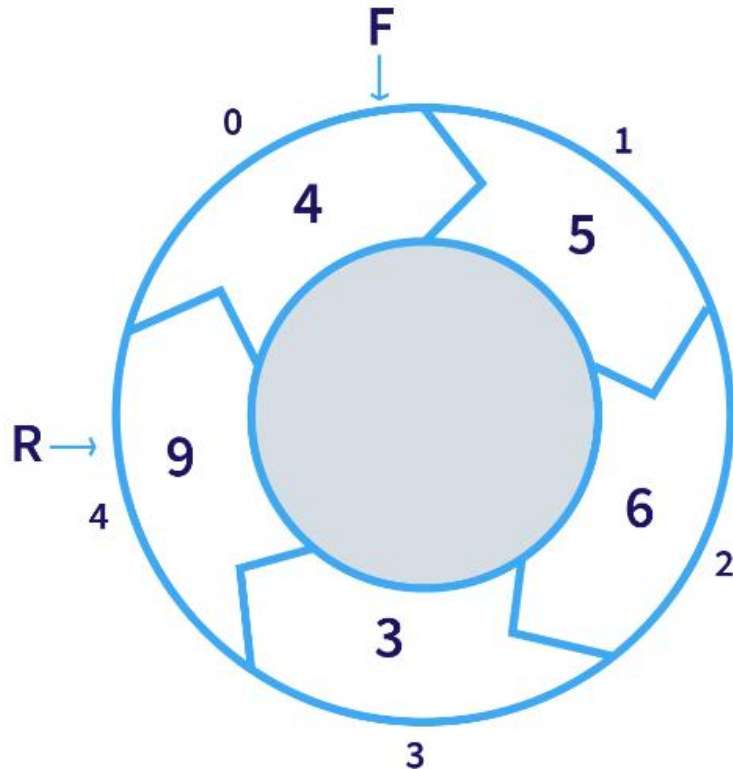
Implementation using linked lists

[QueuewithListMain.c](#) [QueuewithList.c](#) [QueList.h](#)

Data structures and its applications

Circular queue

A circular queue is a linear queue that behaves as if the end position of the queue is connected to the start position, forming a circular ring. It is also known as a Ring Buffer. In array-based implementation, the circular behavior is achieved by manipulating the queue pointers.



Data structures and its applications

Circular Queue ...

The queue pointers (Front (**front**) pointer and Rear (**rear**) pointer) will start moving from 0 to N-1 (Maximum size of Queue = N) and after reaching N-1, they come back to 0, again they start from 0, 1..., up to N-1 and they continue moving around the Queue.

The variable **rear** points to the next free position in the buffer; **front** points to the first full position in the buffer.



Data structures and its applications

Circular Queue ...



When is the buffer empty? When is it full?

The buffer is empty when **rear == front**;

The buffer is full when

$((\text{rear} + 1) \% \text{BUFFER_SIZE}) == \text{front}$.

Implementation of a circular queue using arrays:

[CircularQMain.c](#)

[CircularQueue.c](#)

[CircularQ.h](#)

Implementation using list:

[CQWithListMain.c](#)

[CQWithList.c](#)

[CQWithList.h](#)

Data structures and its applications

Deque



A deque is a linear data structure, which stands for Double Ended Queue. Unlike **queue**, where the data can be only inserted from one end and deleted from another, in a **deque**, the data can be inserted and deleted from both front and rear ends.

Hence Deque inherits the properties of both queues and stacks. Additionally, the implementation of this data structure requires constant time, i.e., time complexity = $O(1)$. This means you can use deque to your advantage to implement both the queue and stack

The insertion and deletion in deque can be limited to one end. When you do that, the deque becomes a stack.

As deque can operate on both ends, you can extract queue properties by limiting insertion at the front node and removal at the rear node. Doing this will allow deque to act as a linear queue.

Data structures and its applications

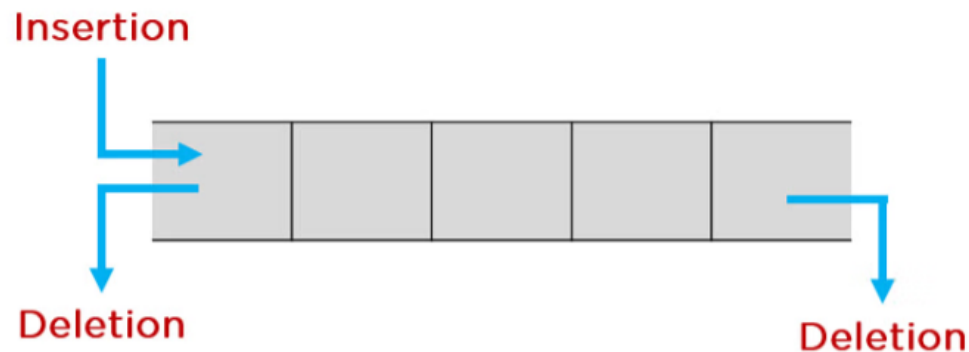
Deque

Types of Deque in Data Structure

There are two types of deque which are created by restricting certain operations.

Input-Restricted Deque:

It is a deque with some limitations while performing insertion operations. In the Input-Restricted Deque, it will perform the deletion at both ends, whereas it performs the insertion at only one end. The image below shows how input restricted deque limits insertion at one end.



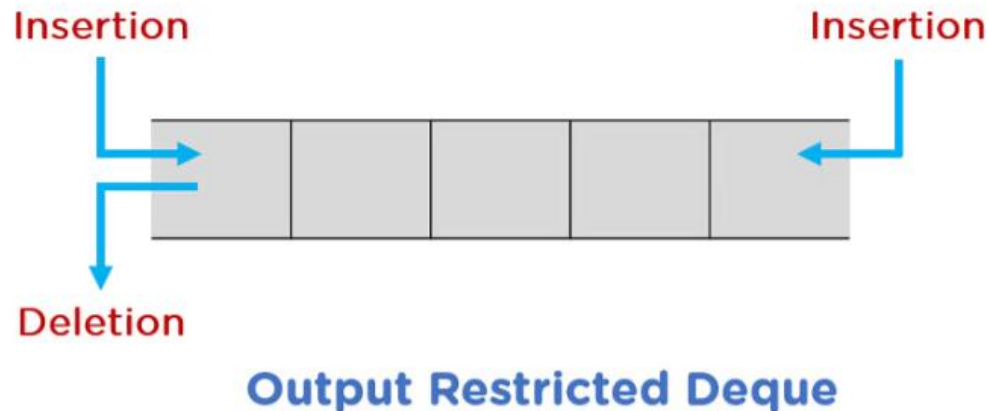
Input Restricted Deque

Data structures and its applications

Deque

Output-Restricted Deque:

It is a deque with some limitations while performing deletion operations. In the Output-Restricted Deque, it will perform the insertion at both ends, whereas it performs the deletion of elements at only one end. The image below shows how output restricted deque limits removal at one end.



Operations on Deque

Four basic operations are performed on deque, they are as follows:

1. Insertion at Rear
2. Insertion at Front
3. Deletion at Front
4. Deletion at Rear

Along with these primary operations, you can also perform `isEmpty()`, `isFull()` and `Peek()` operations. These operations are called supportive queue operations.

The time required to implement all these functions must be constant, i.e., time-complexity = $O(1)$.

Data structures and its applications

Deque

Implementation

Can be done in four different ways:

Array-Based Deque

A fixed-size array is used to store the elements of the deque. Two pointers (or indices) are maintained: one for the front and one for the rear.

Linked List-Based Deque

Utilizes nodes that contain data and pointers to the next and previous nodes.

Elements can be added or removed from either end without needing to copy existing elements.

Data structures and its applications

Deque



Circular Array Deque

This is a variation of the array-based deque where the array is treated as circular. This means that when the end of the array is reached, it wraps around to the beginning.

Dynamic Array Deque

This implementation begins with a small fixed-size array and dynamically resizes it as needed (similar to a typical dynamic array).

Implementation using arrays (regular, non-circular)

[DequeArrayMain.c](#) [DequeArrayFunctions.c](#) [DequeArray.h](#)

Using Lists:

[DequeListMain.c](#) [DequeList.c](#) [DequeList.h](#)

Data structures and its applications

Deque



Applications of the deque in data structure

Palindrome Checker: Deque in data structure can be used to implement the program to check whether a number is a palindrome or not.

Multiprocessor Scheduling: When multiple processes in a system are being carried out by multiple processors like CPU, or Core, at that time that system uses a multiprocessor scheduling algorithm. All the processors use our deque data structure to store all the different threads of processes. We also call this algorithm the A-Steal algorithm for scheduling.

Deque can be also used for implementing both the stack and queue.

Deque can be also used to store the history in the browser.

Deque is widely implemented for the undo operations in software.

Data structures and its applications

Priority Queue



What is Priority Queue?

A priority queue serves as a specialized data structure where elements are organized based on their priority values, ensuring that higher-priority items are processed before lower-priority ones.

This organization enables efficient retrieval of the most critical elements.

Priority is a number. We can define the order the way we like it:

For example: Lower this number, higher the priority

Higher the value of this number, higher is the priority

Sample implementation:

[priorityqMain.c](#)

[priorityq.c](#)

[priorityq.h](#)

Data structures and its applications

Applications of queues

Some of the common application of queue in data structure:

Queues can be used

- to schedule jobs and ensure that they are executed in the correct order.
- to manage the order in which print jobs are sent to the printer.
- to implement the breadth-first search algorithm.
- to manage incoming calls and ensure that they are handled in the correct order.
- to manage the order in which processes are executed on the CPU.
- for buffering data while it is being transferred between two systems. When data is received, it is added to the back of the queue, and when data is sent, it is removed from the front of the queue.



Data structures and its applications

Josephus problem and solution



Josephus Problem : Postulates a group of soldiers surrounded by an overwhelming enemy force. There is no hope of victory without reinforcements. There is one horse available for escape

The soldiers agree to a pact to determine which of them is to escape and seek help. The soldiers form a circle and a number n is picked from a hat. One of the names is also picked from the hat.

Beginning with the soldier whose name is picked , they begin to count clockwise around the circle. when the count reaches n , that soldier is removed from the circle and the count begins with the next soldier.

The process continues so that each time the count reaches n , another soldier is removed from the circle. Any soldier removed from the circle is no longer counted. The last soldier remaining is to take the horse and escape.

Data structures and its applications

Josephus problem and solution

Problem Statement:

There are n people standing in a circle waiting to be executed. The counting out begins at some point in the circle and proceeds around the circle in a fixed direction. In each step, a certain number of people are skipped and the next person is executed. The elimination proceeds around the circle (which is becoming smaller and smaller as the executed people are removed), until only the last person remains, who is given freedom.

Given the total number of persons n and a number k which indicates that $k-1$ persons are skipped and the k th person is killed in a circle. The task is to choose the place in the initial circle so that you are the last one remaining and so survive.

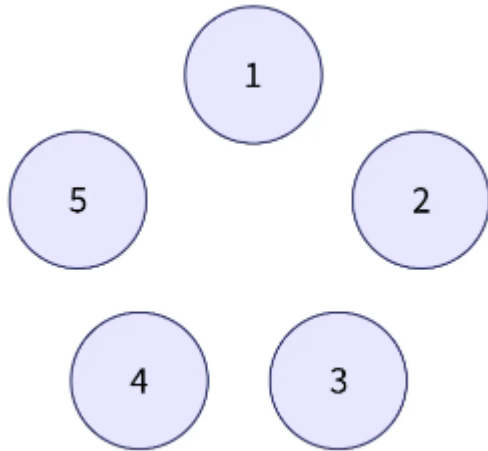


Data structures and its applications

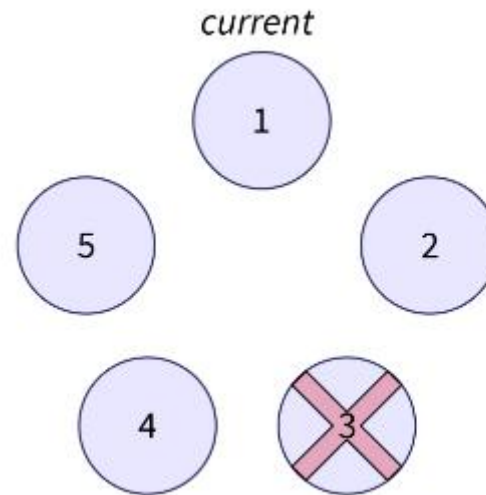
Josephus problem

Example

Initially, 5 people are standing in the circle as follows:



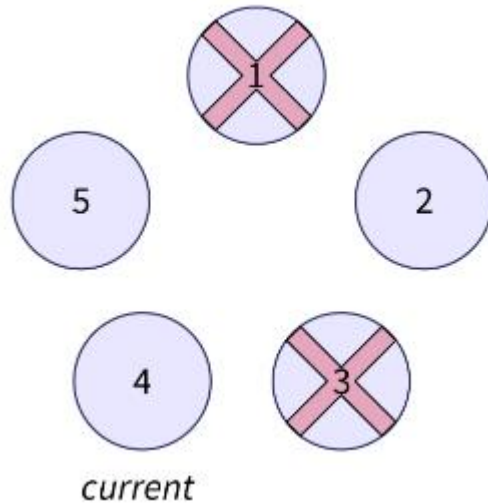
Now starting from 1st position, we will skip $k-1$ people and kill the k th person. k in the example is 3 so the person to be killed is 3rd person.



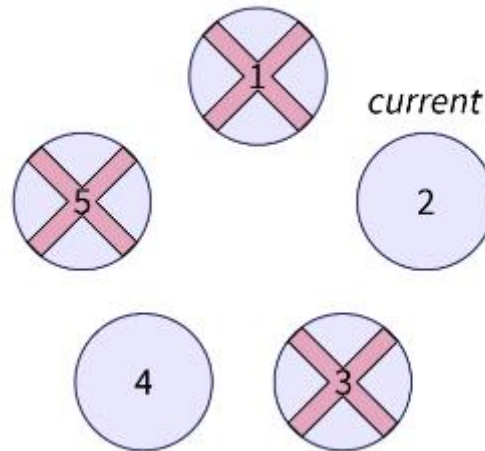
Data structures and its applications

Josephus problem

Next, our current becomes the 4th person in the circle, we go from 4 to 5 and back to 1 which is k th person to be killed.



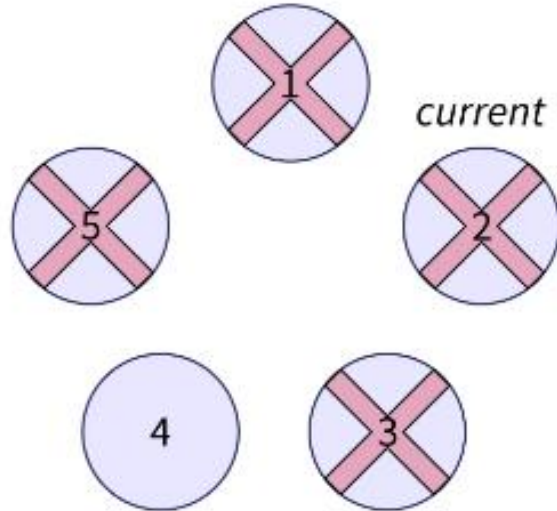
Now the current is 2nd person, we don't consider 3rd person as he is already killed. The next person is 4th and 5th person is killed.



Data structures and its applications

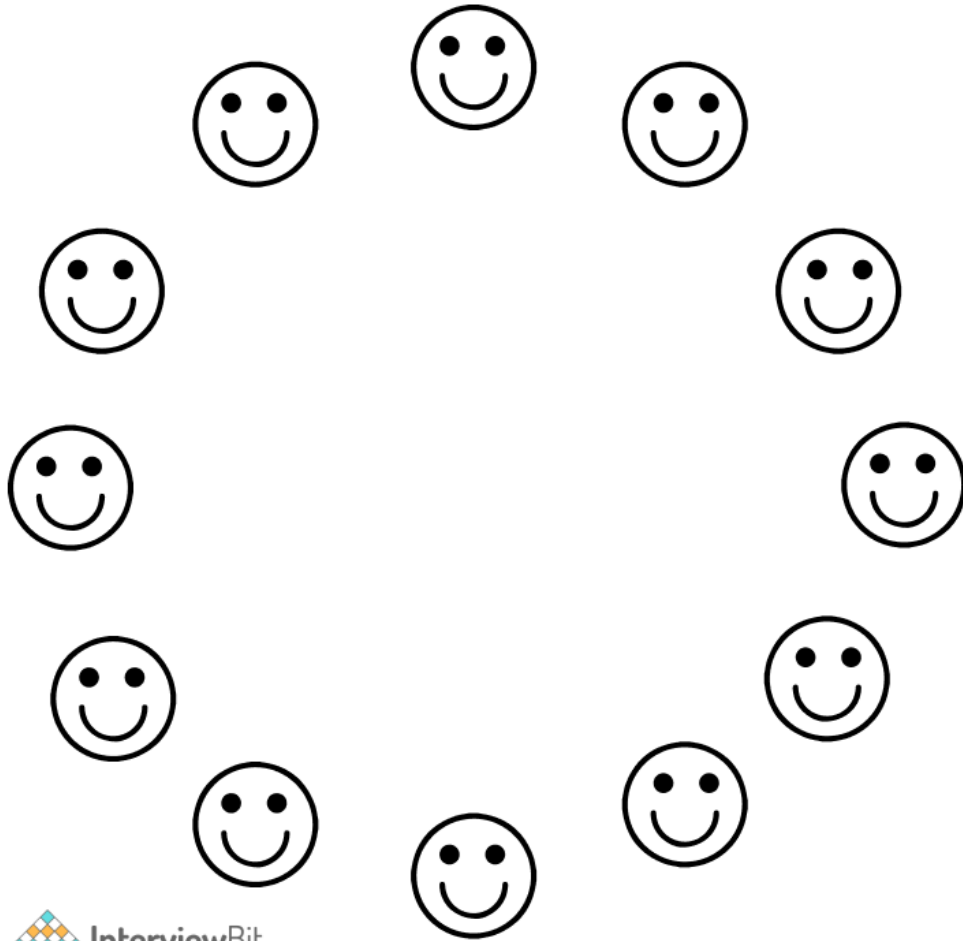
Josephus problem

Again, the current is 2nd person and we go to 4th person and back to 2nd person. Thus, 2nd person is killed in this step. So, the only one who doesn't get killed is the 4th one.



Data structures and its applications

Josephus problem



Data structures and its applications

Josephus problem - Solution



Implementation using an array

.

[JoseArray.c](#)

Implementation using recursion

[JoseRecursive.c](#)

Note:

The problem has the following recursive structure.

$josephus(n, k) = (josephus(n - 1, k) + k - 1) \% n + 1$ and

$josephus(1, k) = 1$

After the first person (kth from the beginning) is killed, n-1 persons are left. Make recursive call for Josephus(n - 1, k) to get the position with n-1 persons. But the position returned by Josephus(n - 1, k) will consider the position starting from $k \% n + 1$. So, make adjustments to the position returned by Josephus(n - 1, k).

Data structures and its applications

Josephus problem - Solution



- The input to the program is the number n and list of names, which is the clockwise ordering of the circle, beginning with the soldier from whom the count is to start.
- The program should print the names in the order that they are eliminated and the name of soldier who escapes.
- For example, if $n=3$ and that there are five soldiers named A,B,C,D and E. We count three soldiers starting at A, so that C is eliminated first.
- We then begin at D and count D E and back to A. A is eliminated. Then we count B D and E, E is eliminated. And finally, B D and B is eliminated.
- D is the one who escapes

Data structures and its applications

Josephus problem - Solution



Pseudo code of implementation using circular list

```
·      read(n)
      read(name)
      while(all the names are read)
      {
          insert name on the circular list
          read(name)
      }
      while(there is more than one node on the list)
      {
          count through n-1 nodes on the list
          print name in the nth node
          delete the nth node
      }
      print the name of the only node on the list
```

Data structures and its applications

Josephus problem - Solution

Implementation using linked list

.

[JosephusList.c](#)



Data structures and its applications

Process scheduling

First Come First Served (FCFS) – Implemented using a Queue

Shortest Job First (SJF) – Priority based. The burst time is used to prioritize and schedule a job/process. The running process may be pre-empted to make way for a shorter job.

Shortest Job First (Non pre-emptive) – Jobs are placed in a queue such that the shortest job takes highest priority. But, the running job is not pre-empted.

Longest Job First (Pre-emptive)

Longest Job First (Non pre-emptive)

Pre-emptive priority based scheduling

Round Robin scheduling

If more than one process has the same priority, then FCFS amongst them



Data structures and its applications

Tree

There are many basic data structures that can be used to solve application problems.

Comparison of Array and a Linked List

Array is a good static data structure that can be accessed randomly and is fairly easy to implement.

Linked List on the other hand is dynamic and is ideal for application that requires frequent operations such as add, delete, and update. One drawback of linked list is that data access is sequential.

Then there are other specialized data structures like, stacks and queues that allows us to solve complicated problems using these restricted data structures.



Data structures and its applications

Tree ...



We can extend the concept of linked data structure (linked list, stack, queue) to a structure that may have multiple relations among its nodes. Such a structure is called a **tree**.

What is a tree?

A tree is a collection of nodes connected by directed (or undirected) edges.

A tree is a *nonlinear* data structure, compared to arrays, linked lists, stacks and queues which are linear data structures.

A tree can be empty with no nodes or a tree is a structure consisting of one node called the **root** and zero or one or more subtrees.

Data structures and its applications

Tree ...

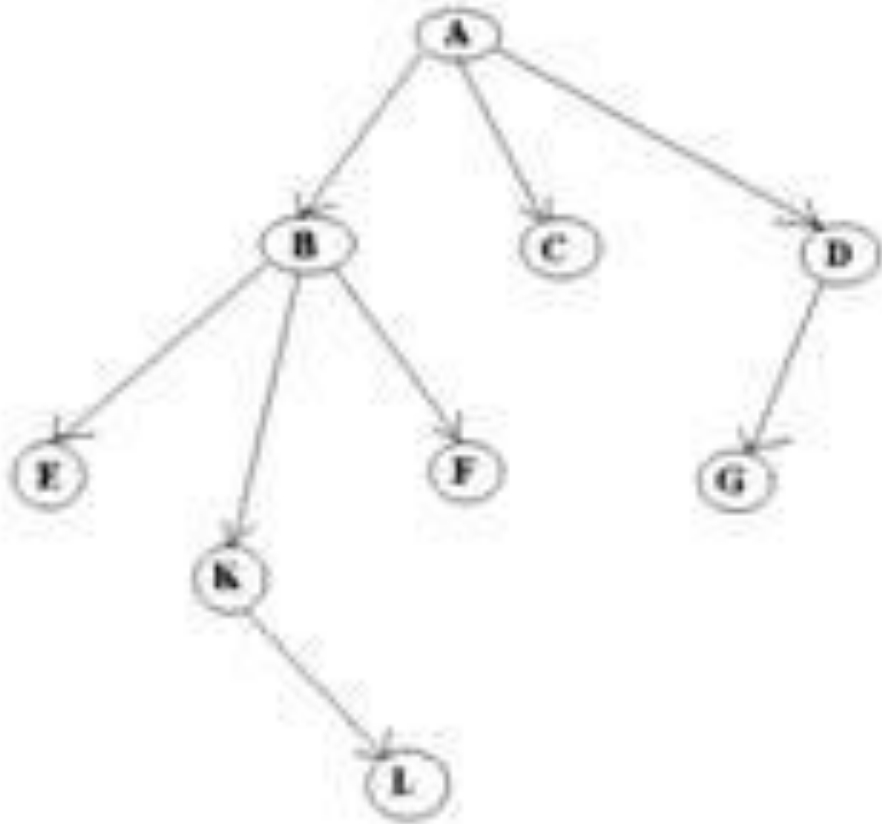
A tree has following general properties:

- One node is distinguished as a **root**;
- Every node (exclude a root) is connected by a directed edge *from* exactly one other node; A direction is: *parent* -> *children*
- Acyclic : Can not start from a node, traverse and come back to the same node
- Connected. Any two nodes are connected by a path which may go through the root.
- A tree of n nodes has $n - 1$ edges – neither more not less.



Data structures and its applications

Tree ...



A is a parent
of B, C, D,

B is called a
child of A.

On the other
hand, B is a
parent of E, F,
K

In the picture, the root
has 3 subtrees.

Data structures and its applications

Tree ...



Terminology

Each node can have *arbitrary* number of children. Nodes with no children are called **leaves**, or **external** nodes. In the picture (previous slide), C, E, F, L, G are leaves.

Nodes, which are not leaves, are called **internal** nodes. Internal nodes have at least one child.

Nodes with the same parent are called **siblings**. In the picture, B, C, D are called siblings.

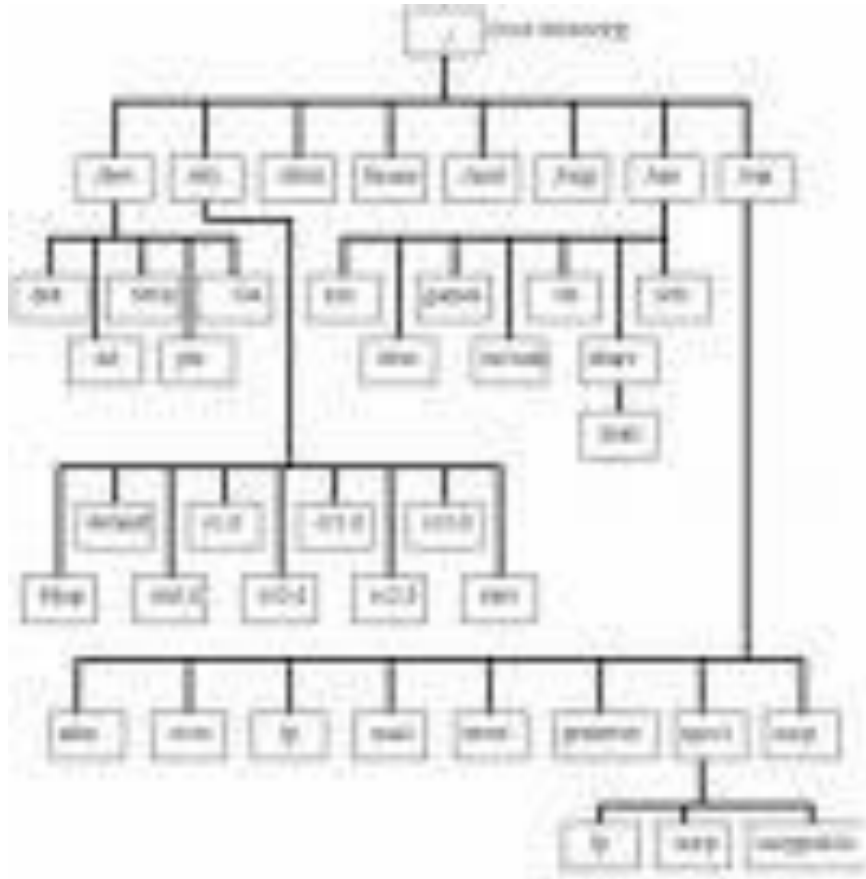
The **depth of a node** is the number of edges from the root to the node. The depth of K is 2.

The **height of a node** is the number of edges from the node to the deepest leaf. The height of B is 2.

The **height of a tree** is the height of root.

Data structures and its applications

General Tree



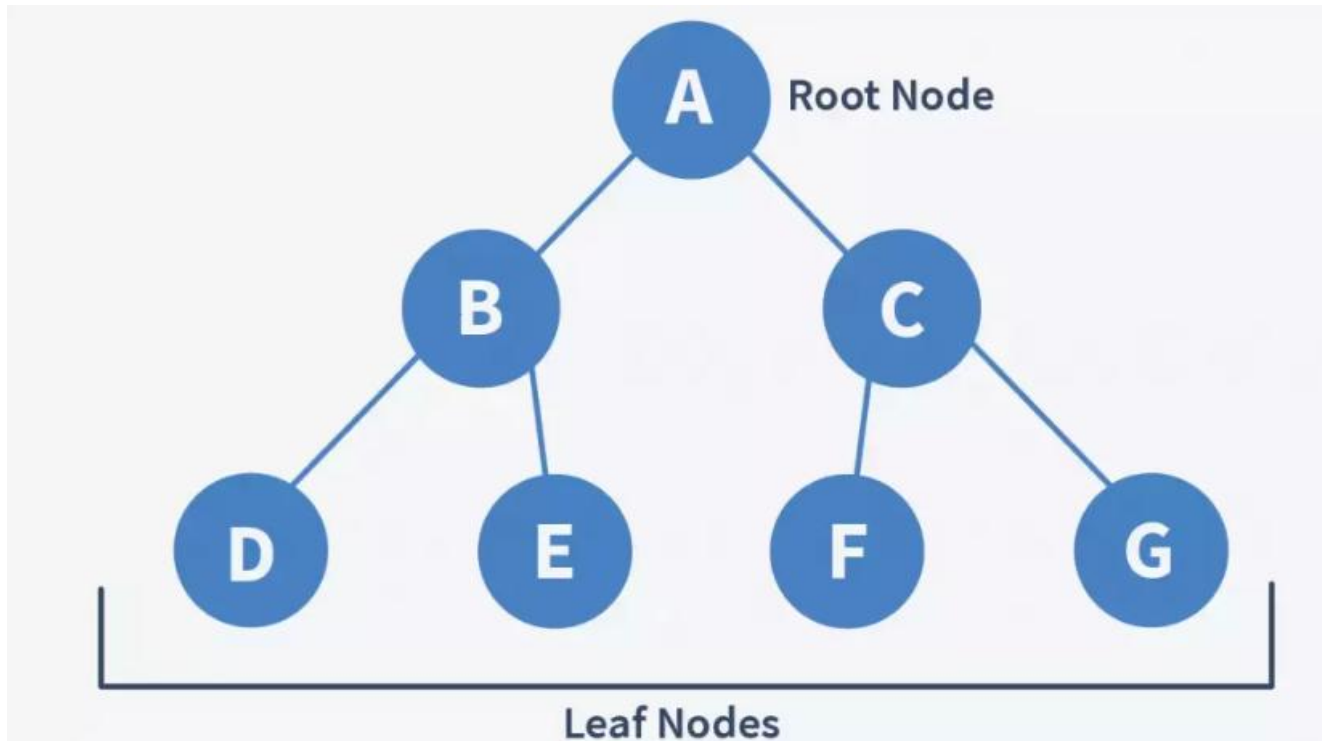
Data structures and its applications

Binary Tree

What is a Binary Tree?

- A binary tree is a specialized version of the General tree. A binary tree is a tree in which each node can have at most two nodes.

The node at the top of the hierarchy of a tree is called the root node.

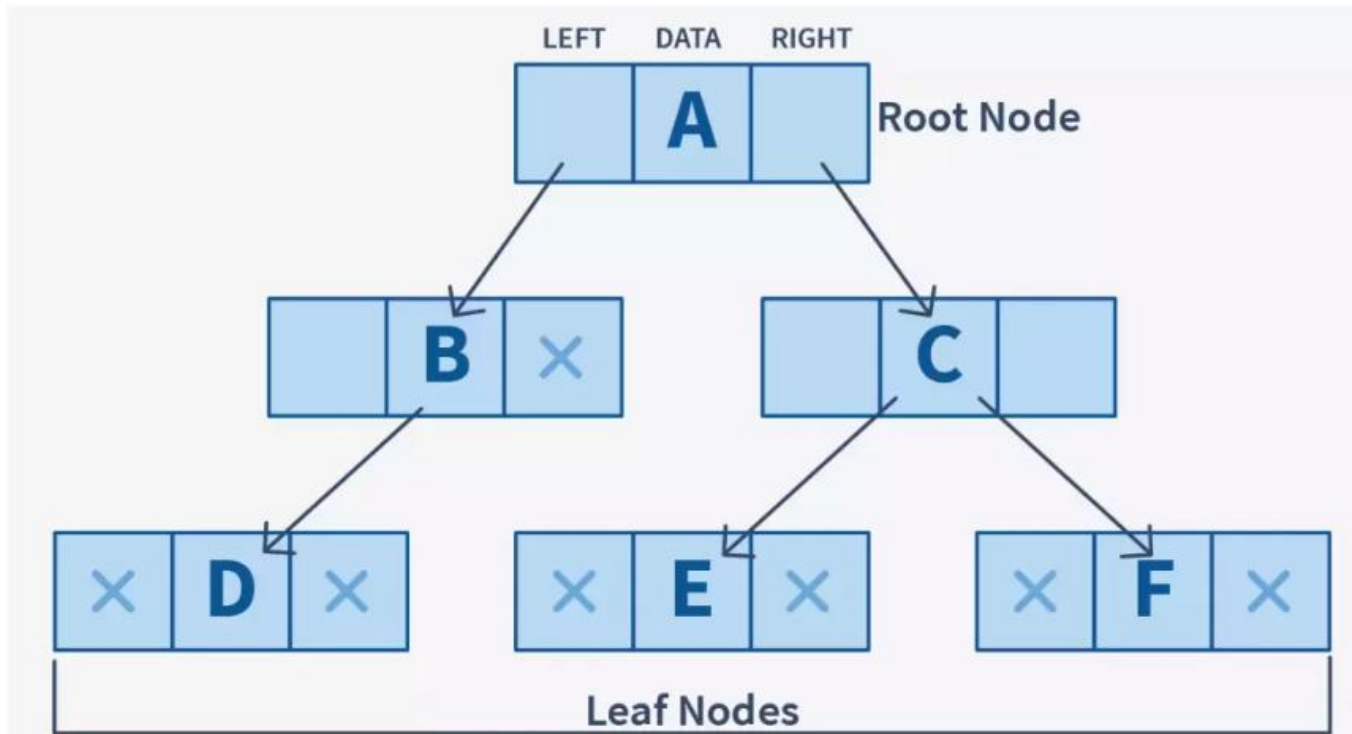


Data structures and its applications

Binary Tree ...

Each node in a binary tree has these three elements –

- Data item that is the data stored in the node
- Address of the left child
- Address of the right child



Data structures and its applications

Binary Tree ...



Terminology in Binary Tree

Nodes – Nodes are the building blocks of any data structure. They majorly contain some data and link to the next/previous nodes. In the case of binary trees, they contain the address of the left and the right child respectively.

Root – The topmost node in a tree is known as the root node. A tree can have at most one root node.

Parent Node – A node (except the root) that has a succeeding node is known as a parent node.

Child Node – A node that has a preceding node is known as a child node. A node can be both parent and child depending on the node that is in context.

Leaf Node – A node with no children.

Internal Node – A node that has at least one child node is known as an internal node.

Depth of a Binary Tree – The number of edges from a node in the tree to the root node.

Height of a Binary Tree – The number of edges from the deepest node in the tree to the root node.

Data structures and its applications

Binary Tree ...

Level of a node

- Level of a node represents the number of connections between the node and the root. It represents generation of a node. If the root node is at level 0, its next node is at level 1, its grand child is at level 2 and so on. Levels of a node can be shown as follows:

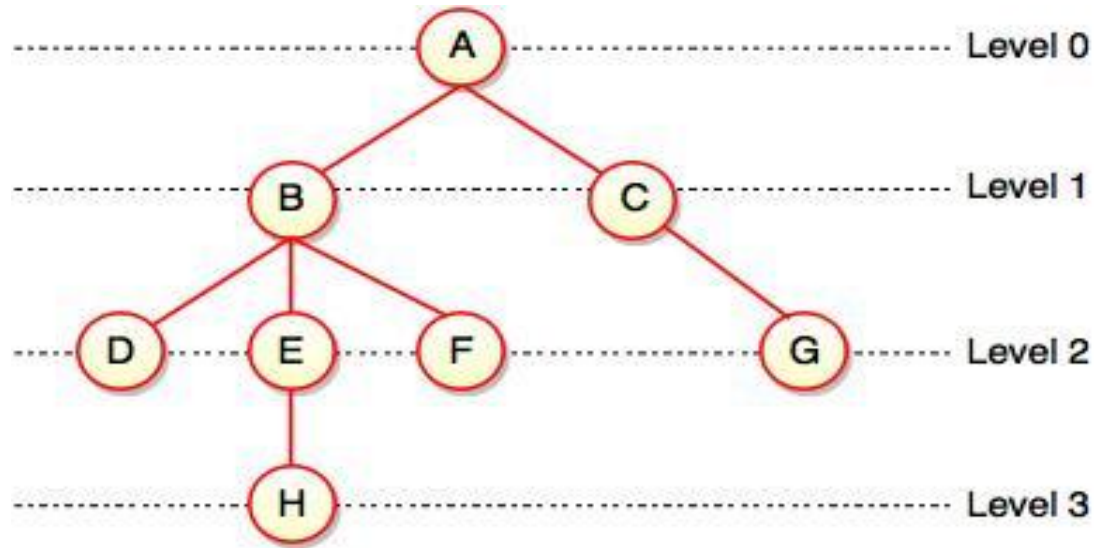


Fig. Levels of Tree

Data structures and its applications

Binary Tree ...

Degree of Node

- Degree of a node represents a number of children of a node.

Edge

Edge is a connection between one node to another. It is a line between two nodes or a node and a leaf.

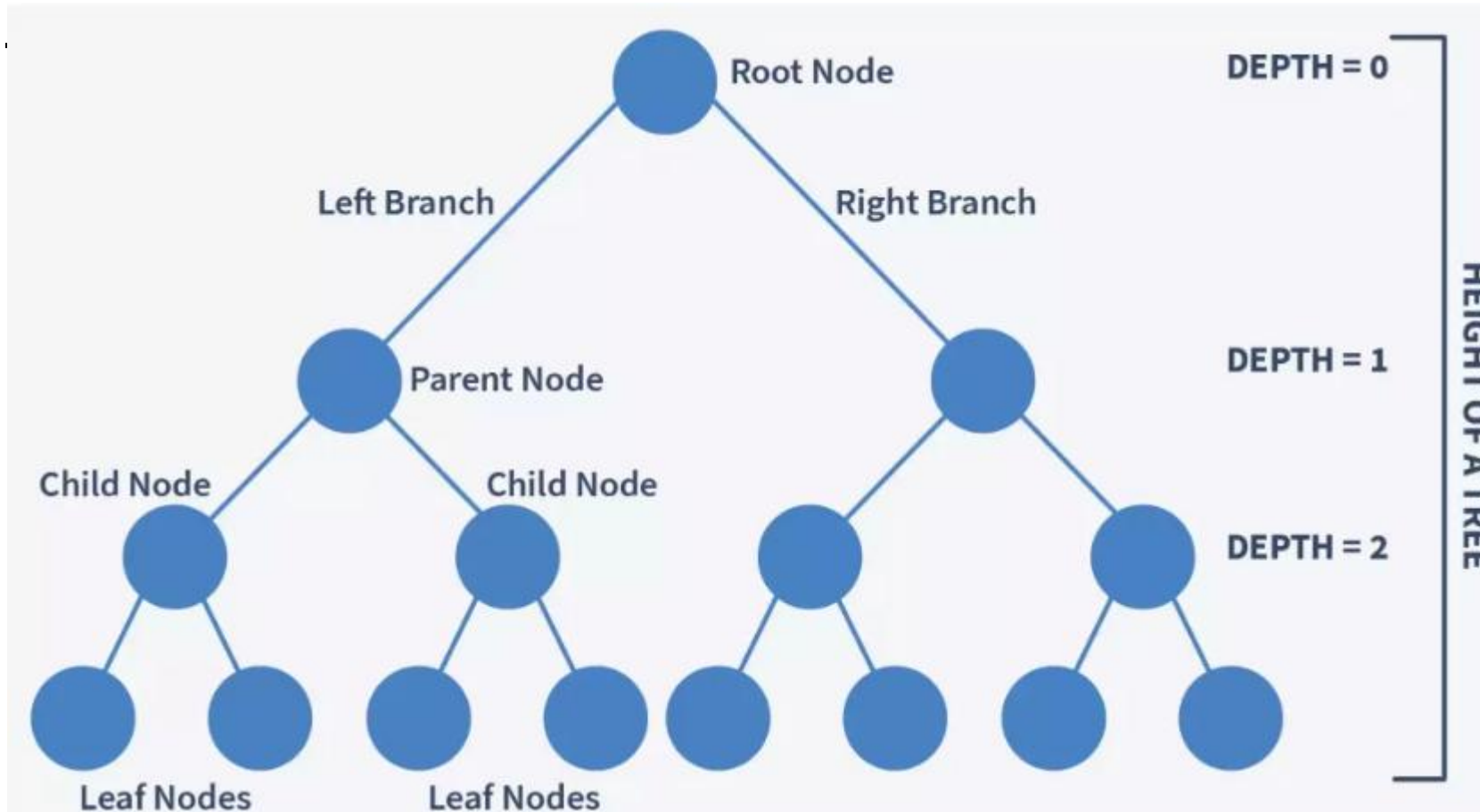
Path

Path is a number of successive edges from source node to destination node



Data structures and its applications

Binary Tree ...



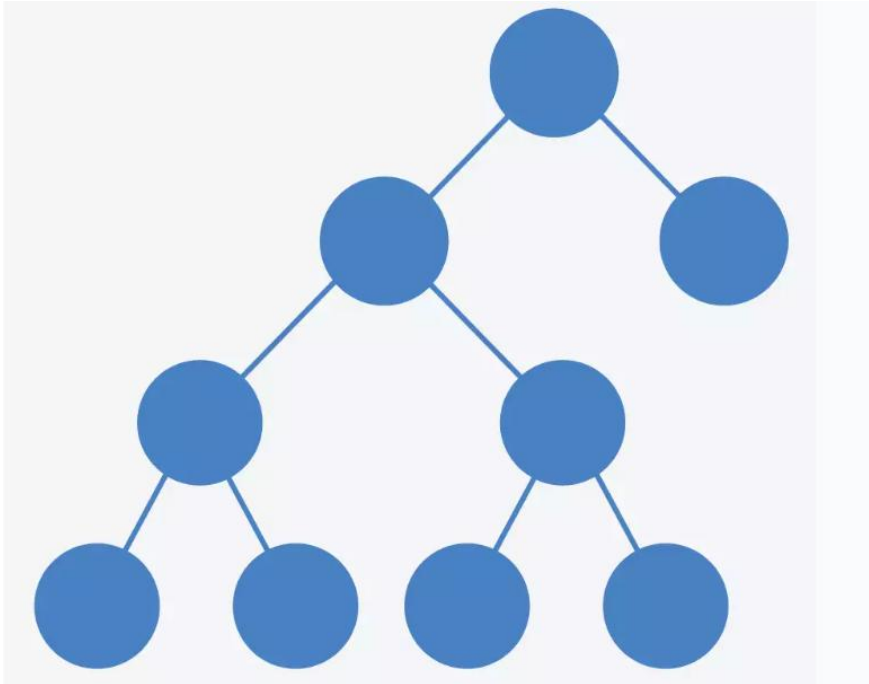
Data structures and its applications

Types of Binary Tree

Full Binary Tree

A full binary tree, also known as a proper binary tree, is a tree in which each internal node has **either zero or two** child nodes.

In other words, if in a binary tree a node contains only one child node, it is not a full binary tree.



In a full binary tree, if there are n number of total nodes –

- The number of internal nodes is given by $(n-1)/2$
- The number of leaf nodes is given by $(n+1)/2$

Complete Binary Tree

A complete binary tree is a binary tree in which all the elements are arranged without missing any sequence.

In a complete binary tree –

All the levels are completely filled except the last level that may or may not be completely filled.

Elements are filled from left to right.

The trees shown in the next slide are complete binary trees since they have no empty spaces in them.

Data structures and its applications

Types of Binary Tree ...



Data structures and its applications

Types of Binary Tree ...

Perfect Binary Trees

If in a tree, all the internal nodes have exactly two children nodes, it is known as a perfect binary tree.

In a perfect binary tree, all the leaf nodes are on the same level.

Consider a perfect binary tree with height h , the total number of nodes in this case is given by $2^h - 1$.



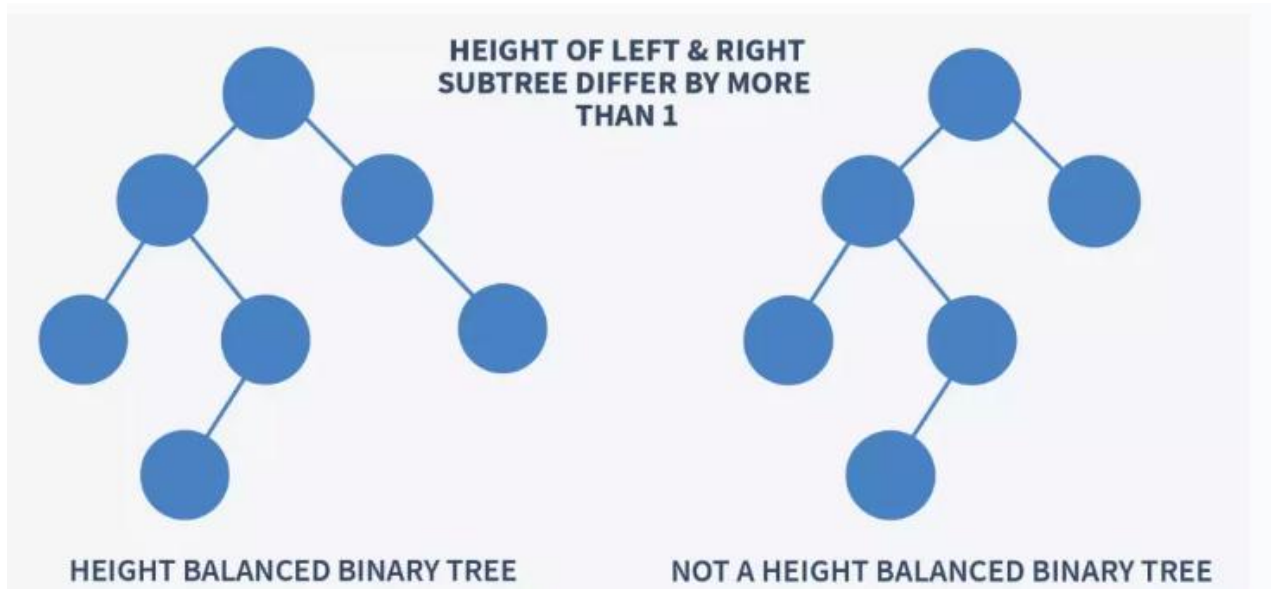
Data structures and its applications

Types of Binary Tree ...

Balanced Binary Trees

A binary tree is said to be balanced if the height of the left and the right subtrees differ by 0 or 1.

In a balanced binary tree, both the left and right trees are also balanced.
The following diagram shows a balanced binary tree –



Data structures and its applications

Types of Binary Tree ...

Notice that in the first image, the right subtree is at the height of 3, and the left subtree is at 2. The difference between the heights of the left and the right subtree is therefore 1. Therefore, the given tree is a balanced binary tree.

In the second image, the right subtree is at the height of 3, whereas the left subtree is at 1. The difference between the heights of the left and the right subtrees is 2. It violates the condition of a balanced binary tree. Hence, the given tree is not a balanced binary tree.



Data structures and its applications

ADT for a tree

A possible ADT for a tree

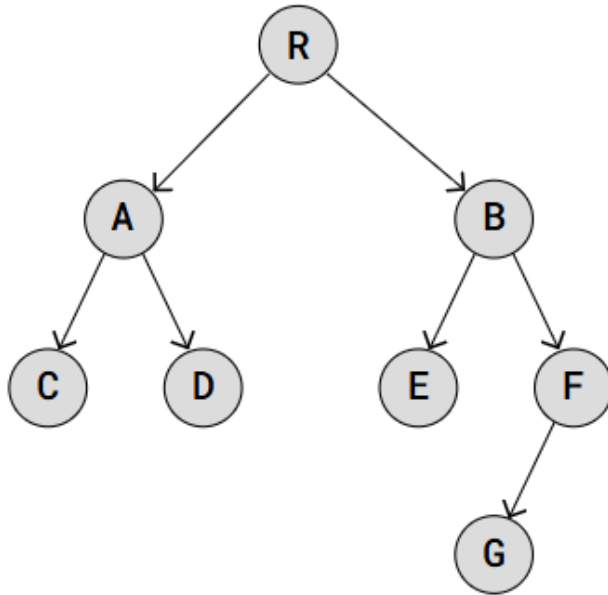
- Create a tree
- Destroy the tree
- Check empty
- Check full
- Add a node
- Remove a node
- Traverse a tree
- Find root
- Find children
- Check whether leaf
- Find height of the tree
- Find the path from the root to a given node



Data structures and its applications

Representation of binary tree in C

A binary tree can be represented using a linked list and also using an array.



How do you represent this tree using arrays?

Data structures and its applications

Representation of binary tree in C



Array Representation :

The size of the array required to store a binary tree depends entirely on the height of the tree (h), where height is defined as the number of edges from the deepest node in the tree to the root node..

To ensure every node can find its mathematically calculated index without throwing an out-of-bounds error, the array must be large enough to accommodate the worst-case scenario: a **fully complete binary tree** of that height.

- For 0-Based Indexing: The maximum index occupied by a node is $(2^{h+1} - 2)$. Therefore, the required array size to prevent out-of-bounds access is : $2^{h+1}-1$.
- For 1-Based Indexing: The maximum index occupied by a node is $(2^{h+1} - 1)$. Therefore, the required array size is : 2^{h+1}

Data structures and its applications

Representation of binary tree in C



0-Based Indexing

Quick reference table

Tree height	Max leaves level	Total nodes (if complete)	Array size required
0 (Only root)	1 node	1	1
1	2 nodes	3	3
2	4 nodes	7	7
3	8 nodes	15	15
h	2^h nodes	$2^{h+1} - 1$	$2^{h+1} - 1$

Data structures and its applications

Representation of binary tree in C



0-Based Indexing (Most Common)

If the root of the tree is stored at index 0:

Left Child of a node at index i is at index: $2 * i + 1$

Right Child of a node at index i is at index: $2 * i + 2$

Parent of a node at index i is at index: $\text{floor}((i - 1) / 2)$

1-Based Indexing

If the root of the tree is stored at index 1

Left Child of a node at index i is at index: $2 * i$

Right Child of a node at index i is at index: $2 * i + 1$

Parent of a node at index i is at index: $\text{floor}(i / 2)$

Array = ['R', 'A', 'B', 'C', 'D', 'E', 'F', None, None, None, None, None, None, 'G', None]

Data structures and its applications

Representation of binary tree in C

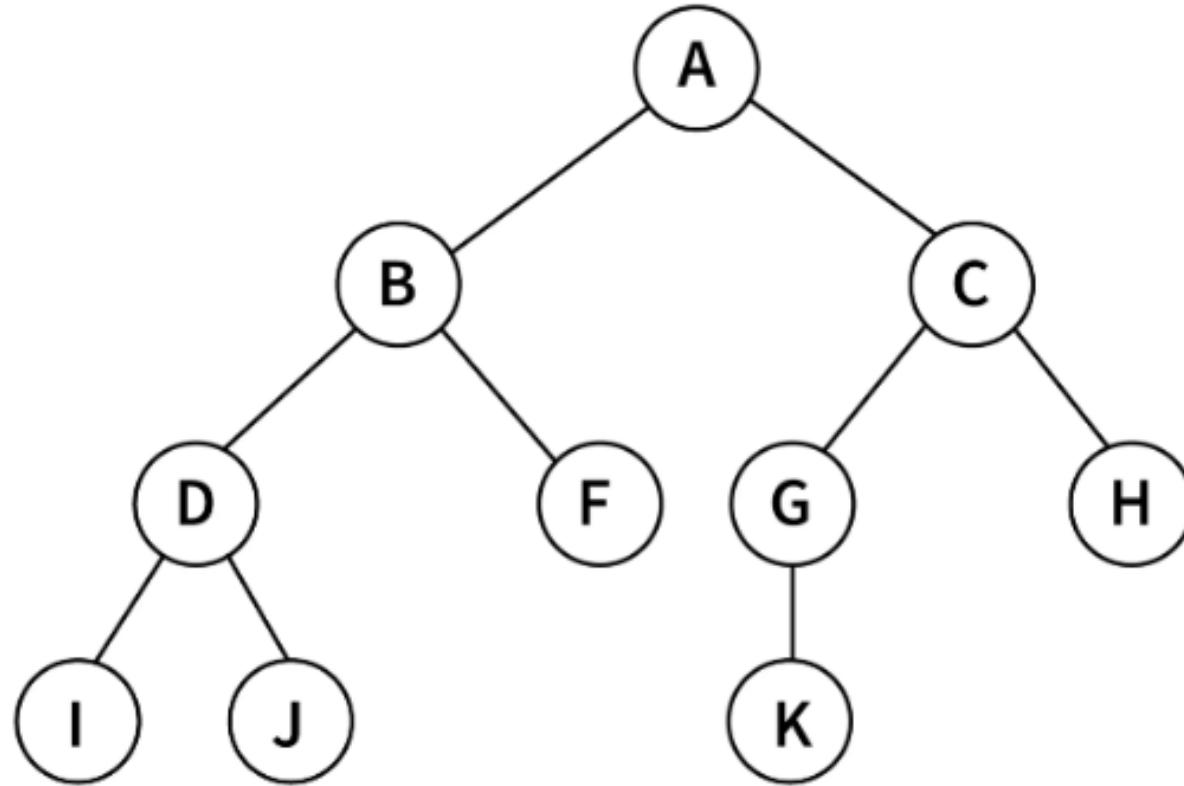
Linked List Representation :

For the linked list representation, we will use the doubly linked list, which has two pointers so that we can point to the left and right children of a binary tree node. NULL is given to the pointer as the address when no child is connected.

Left Child Address	Data	Right Child Address
-----------------------	------	------------------------

Data structures and its applications

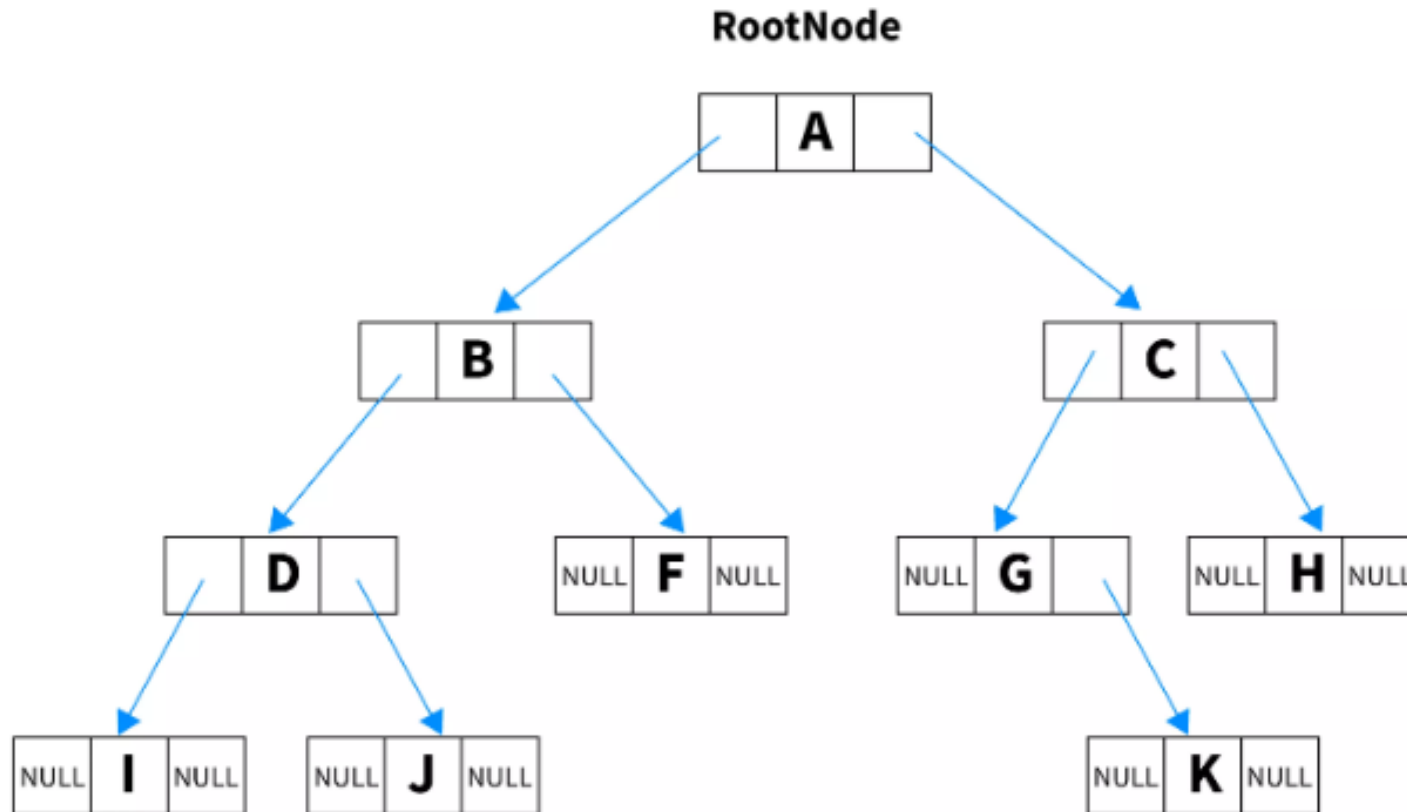
Representation of binary tree in C



Data structures and its applications

Representation of binary tree in C

The linked list representation for the given tree is as shown below:



Data structures and its applications

Tree traversal

Traversal of a Binary Tree

We can traverse the element tree and can display these elements in three formats/methods :

In every traversal method of the binary tree, we need to visit the left node, the right node, and the data of the present/current node(root). There are three methods which have these three actions in a different order.

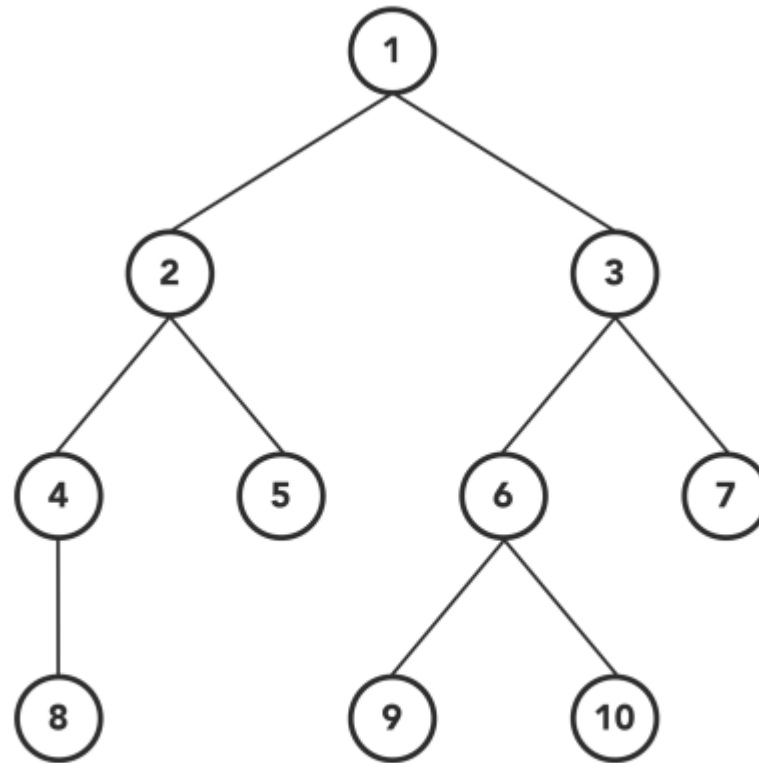


Data structures and its applications

Tree traversal

Pre-Order :

In this type of traversal, first, we will take the value of the current node(present node), then we will go to the left node, and after that, we will go to the right node.

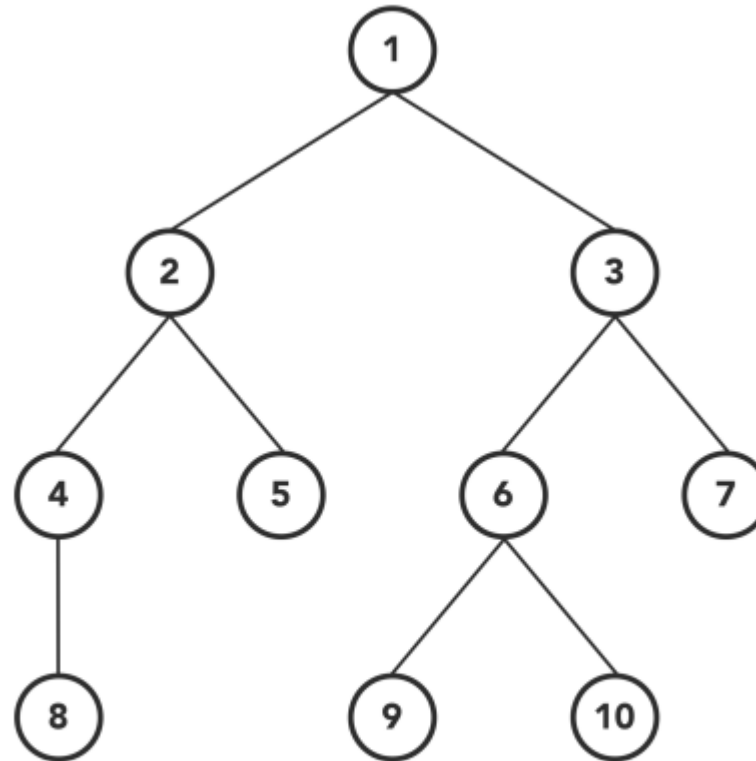


Data structures and its applications

Tree traversal ...

In-Order :

In this type of traversal, first, we'll go to the left node, then we'll take the value of the current node, and then we will go to the right node.

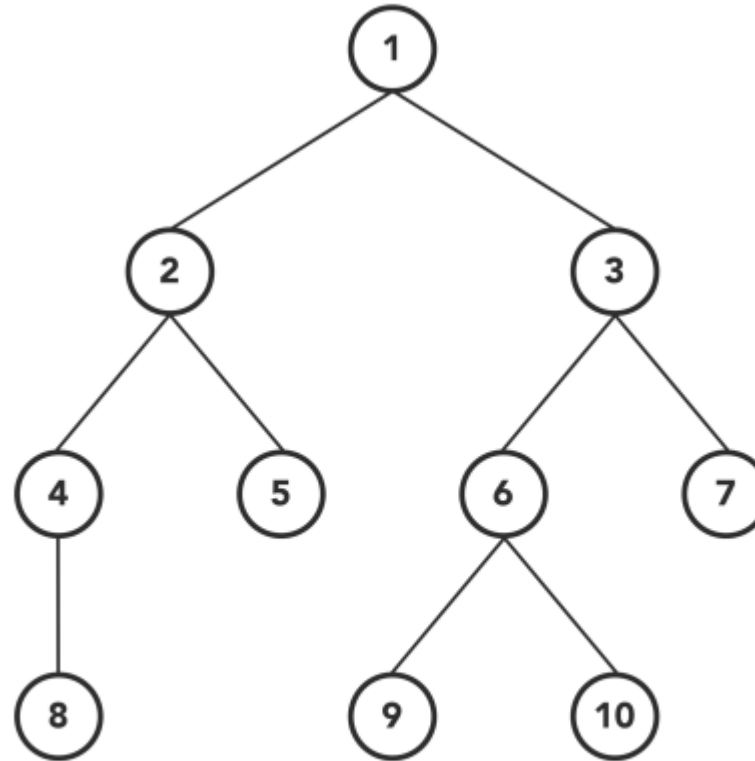


Data structures and its applications

Tree traversal ...

Post-Order :

In this type of traversal, first, we'll go to the left node, then to the right node, and at last, we will take the current node's value.



Data structures and its applications

Tree traversal ...

A sample implementation of a Binary Tree including all three modes of traversal is here:

[BTreeMain.c](#)

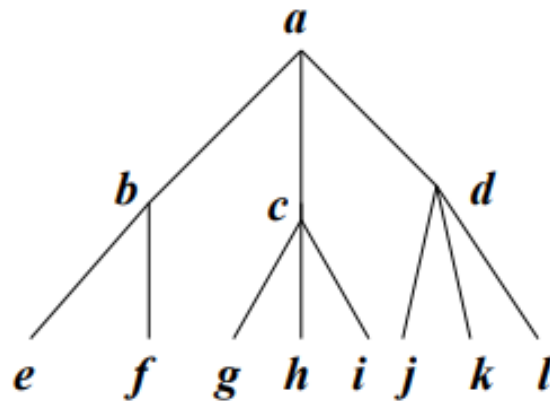
[BTreeTraversal.c](#)

[BTreeTraversal.h](#)



Ordered tree

An ordered tree is a rooted tree in which the set of children of each vertex is assigned a total order. In a drawing of an ordered rooted tree, with the root shown at the top, the children of a vertex are shown from left to right.



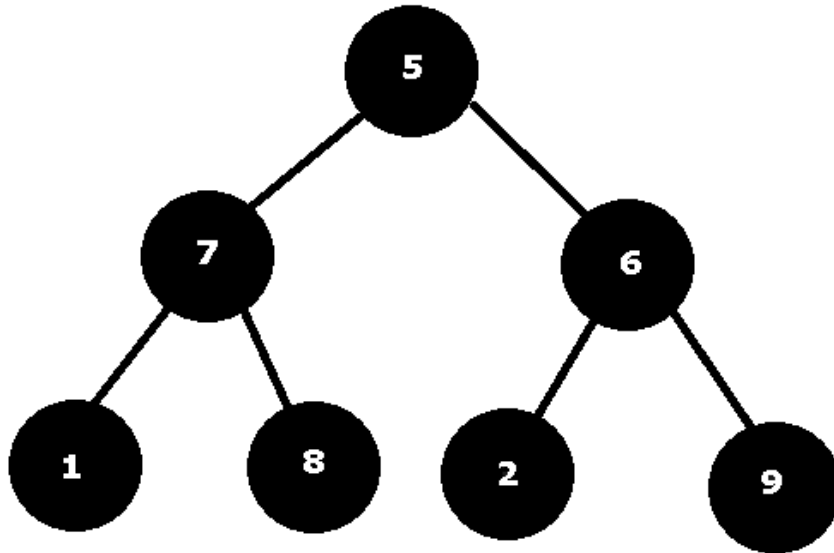
We can call *b* as the first or the oldest child of *a* and *d* as the last or the youngest child of *a*.

Data structures and its applications

Tree traversal ...

Unordered tree

A tree in which nodes are not in any particular order is called an unordered tree



Data structures and its applications

N-ary tree

An n-ary tree is a tree data structure where each node can have at most n children.

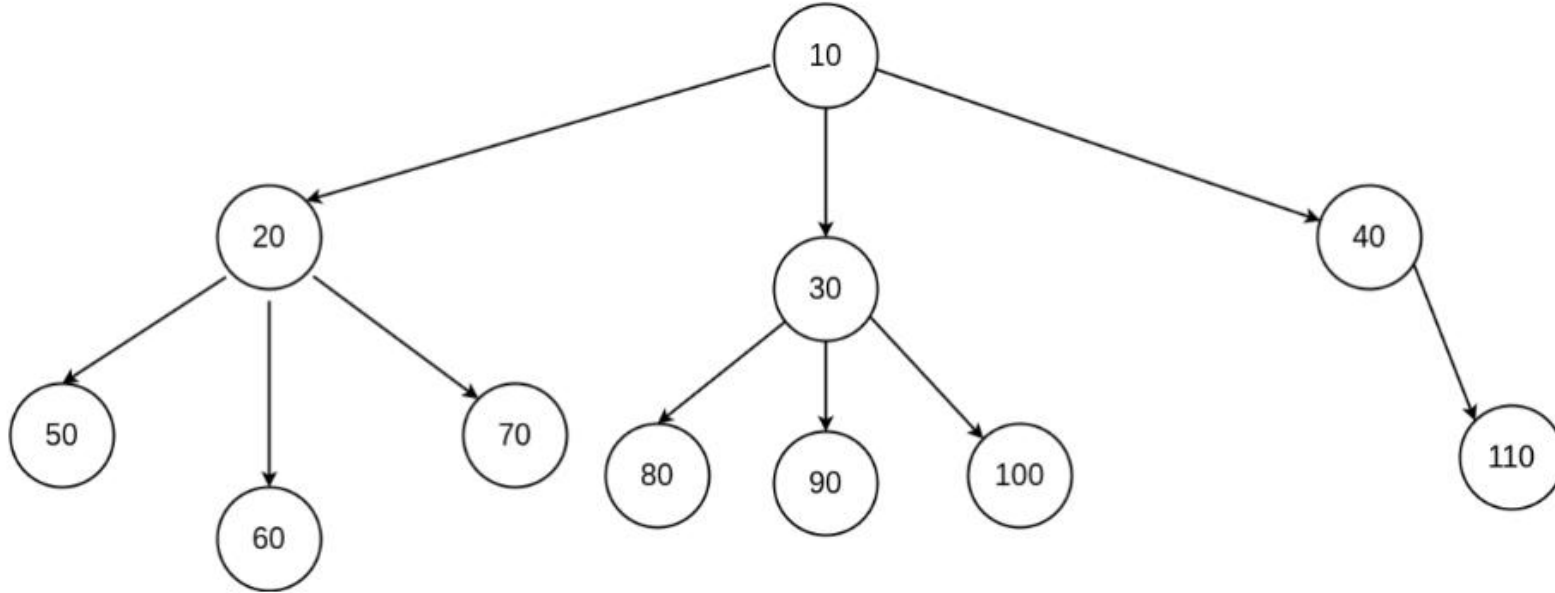
Unlike binary trees, which have at most two children per node, n-ary trees allow for a variable number of children.

They are used in scenarios where each node may have multiple possible paths or branches, such as in hierarchical data structures or decision trees.



Data structures and its applications

N-ary tree ...



In the above figure, we can see that there is one root node with some children. Here we have decided to keep our n value as 3. So, the number of children each node has is less than or equal to 3. This tree can also be called a 3-Ary tree.

Data structures and its applications

N-ary tree ...

Representation in C

First representation

struct node

```
{  
    int info;  
    struct node *child [MAX];  
};
```

Each node can have a maximum of MAX children.

Data structures and its applications

N-ary tree ...

Alternate representation

struct node

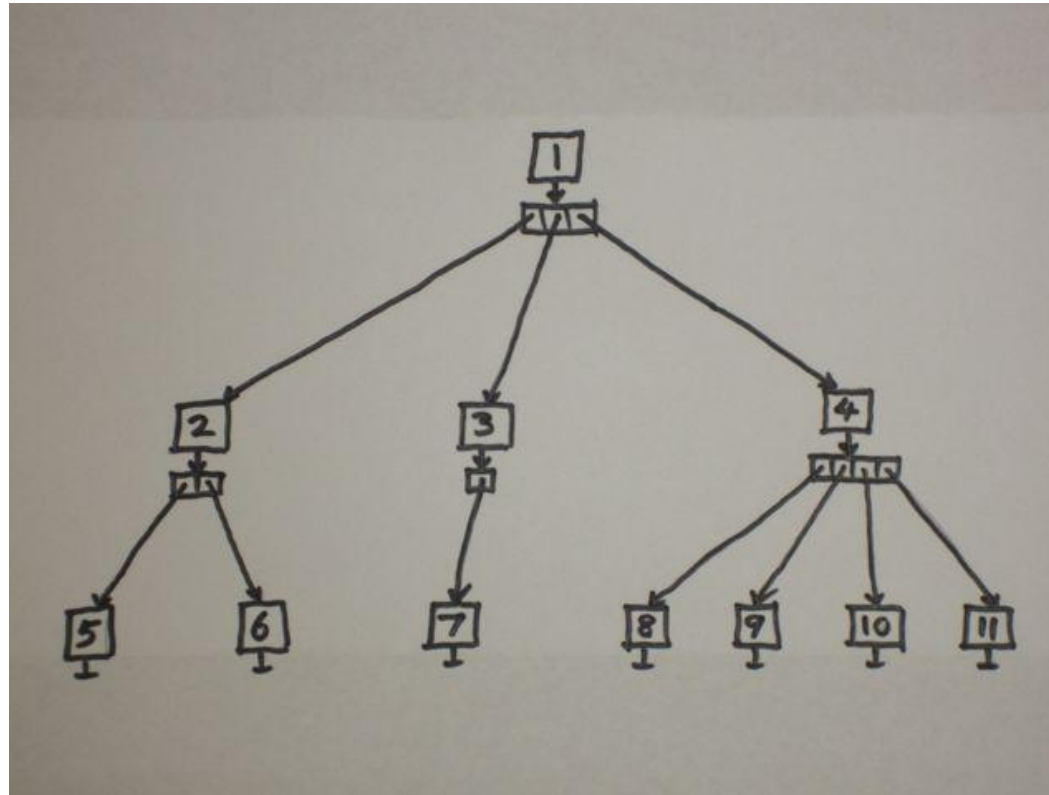
{

int n;

int numKids;

struct node **kids;

};

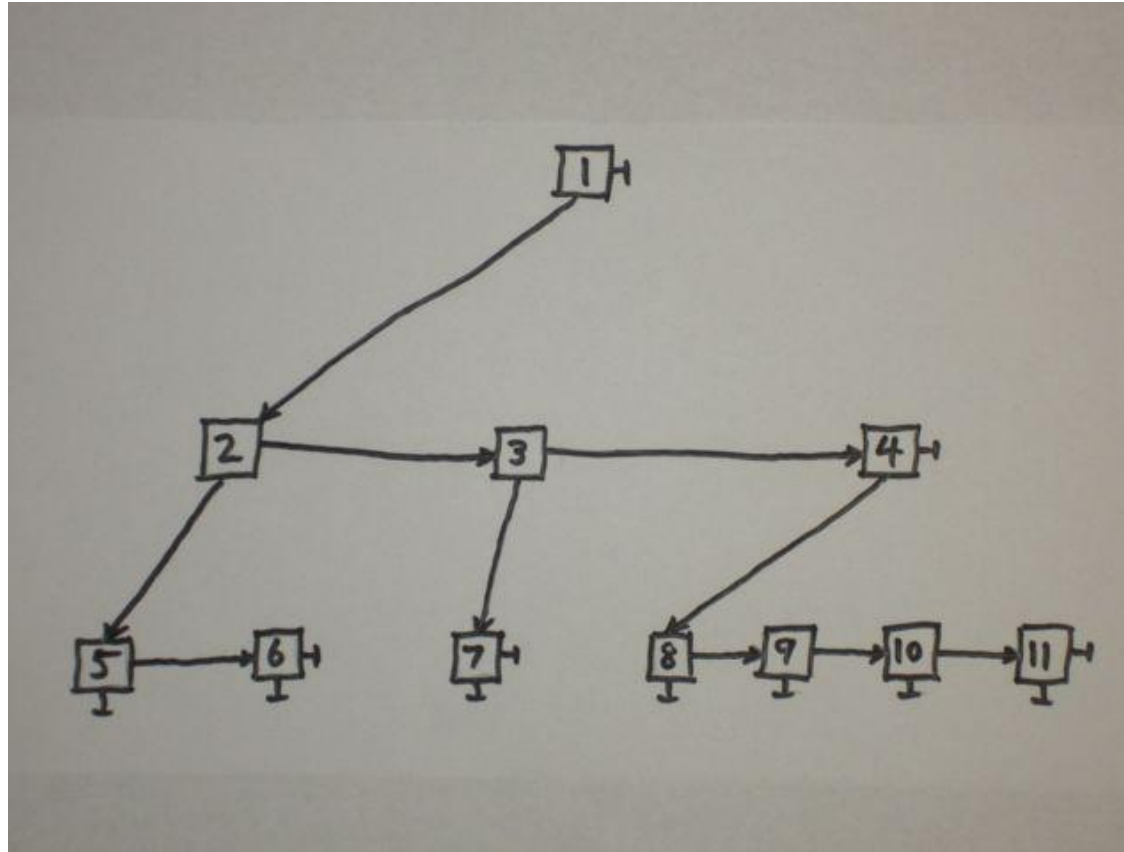


Data structures and its applications

N-ary tree ...

Another representation

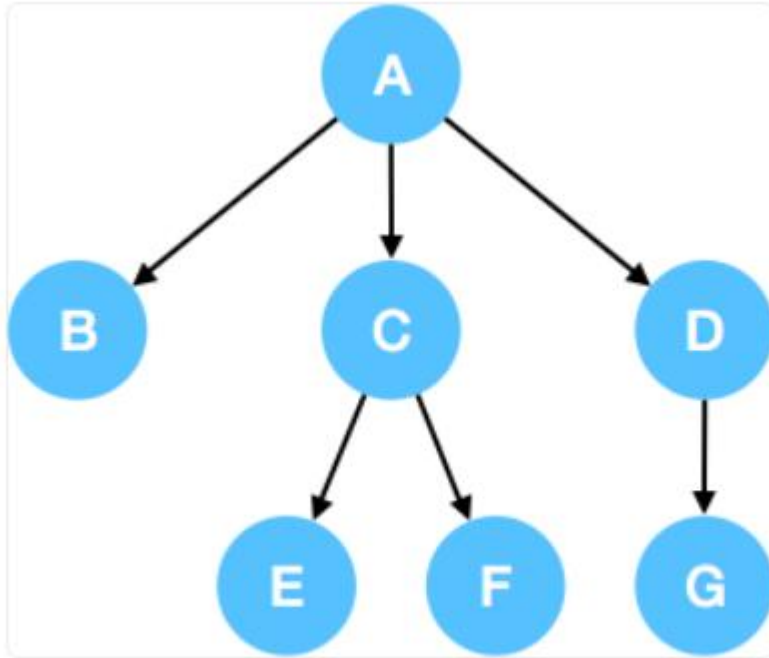
```
struct node  
{  
    int n;  
    struct node *kids;  
    struct node *siblings;  
};
```



From any single node you can only directly access its first kid, through the kids pointer. You access the second kid through kids->siblings, the third kid through kids->siblings->siblings, and so on. In other words, instead of storing a node's kids in an array, you store them in a list; it just so happens that this list can be embedded elegantly in the nodes themselves.

Data structures and its applications

n-ary tree traversal



Pre order Traversal

In an N-ary tree, preorder means visit the root node first and then traverse the subtree rooted at its children one by one. For instance, the preorder of the 3-ary tree above is: A->B->C->E->F->D->G.

Post order Traversal

In an N-ary tree, postorder means traverse the subtree rooted at its children first and then visit the root node itself. For instance, the postorder of the 3-ary tree above is: B->E->F->C->G->D->A.

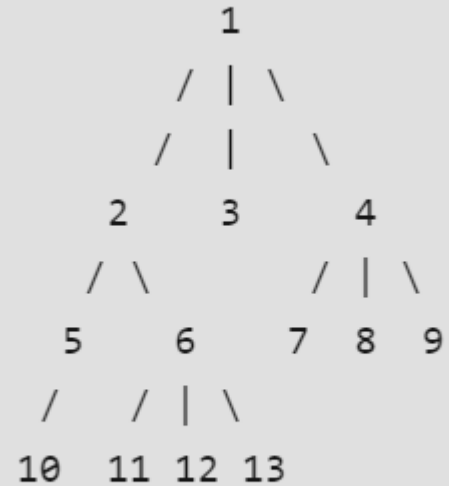
Level-order Traversal

Level-order traversal in an N-ary tree is the same with a binary tree. Typically, when we do breadth-first search in a tree, we will traverse the tree in level order. For instance, the level-order of the 3-ary tree above is: A->B->C->D->E->F->G.

Data structures and its applications

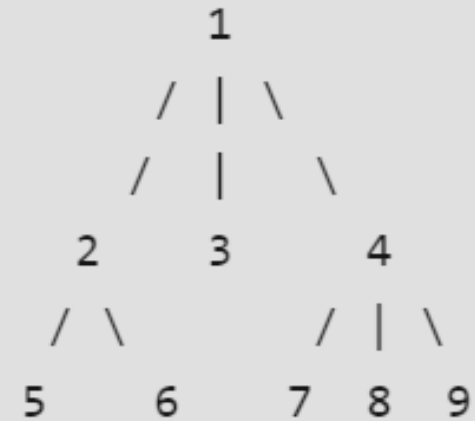
n-ary tree traversal

Input: 3-Array Tree



Output: 1 2 5 10 6 11 12 13 3 4 7 8 9

Input: 3-Array Tree



Output: 1 2 5 6 3 4 7 8 9

Data structures and its applications

n-ary tree traversal



Using the following representation of the tree:

```
.  
struct node  
{  
    int data;  
    struct node *kids;  
    struct node *siblings;  
};
```

pre order traversal

```
void preorder (NODE *root)  
{  
    if (root != NULL)  
    {  
        printf ("Data is %d\n", root->data);  
        preorder (root->kids);  
        preorder (root->siblings);  
    }  
}
```

Data structures and its applications

n-ary tree traversal ...



in order traversal

```
void inorder (NODE *root)
{
    if (root != NULL)
    {
        inorder (root->kids);
        printf ("Data is %d\n", root->data);
        inorder (root->siblings);
    }
}
```

post order traversal

```
void postorder (NODE *root)
{
    if (root != NULL)
    {
        postorder (root->kids);
        postorder (root->siblings);
        printf ("Data is %d\n", root->data);
    }
}
```

Data structures and its applications

Conversion of an n-ary tree to a binary tree

To convert an N-ary tree (generic tree) into a Binary Tree, the standard and most efficient approach is the Left-Child Right-Sibling (LCRS) representation. This method provides a lossless, one-to-one conversion that maps a structure with arbitrary branching into a standard binary tree layout.

The Core Rules

The transformation follows three straightforward structural rules:

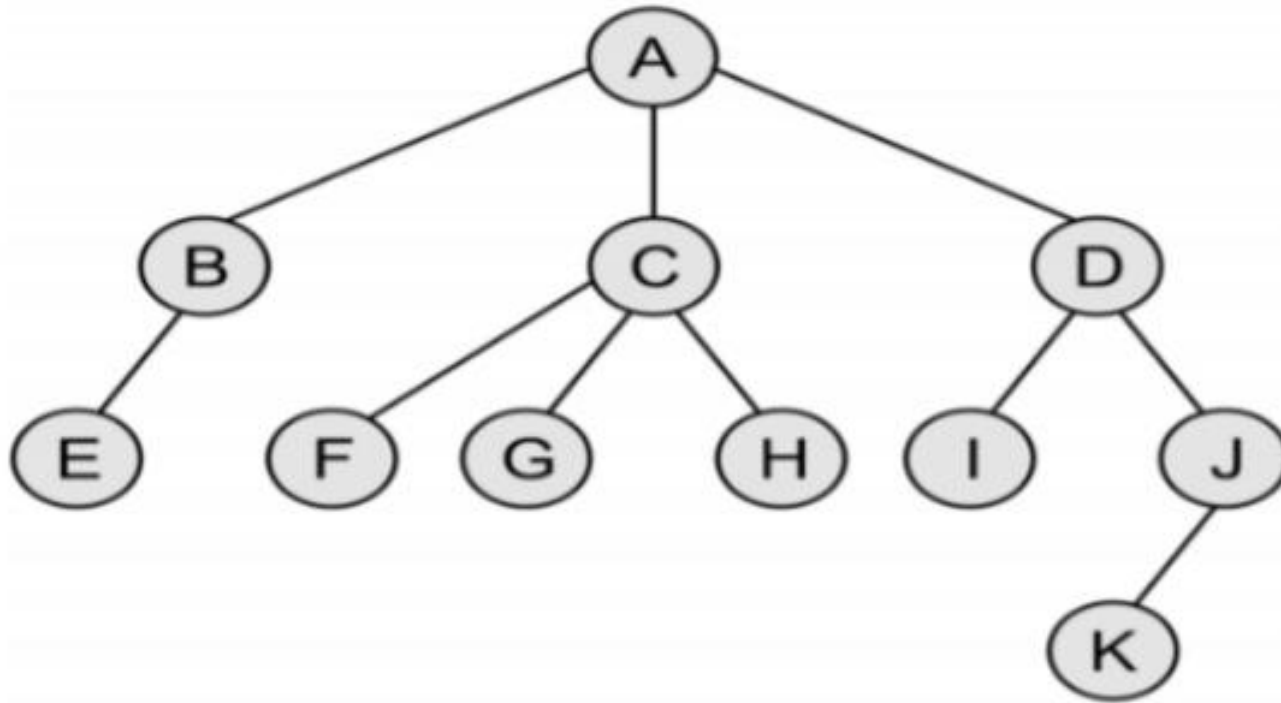
- Root Rule: The root of the Binary Tree is the same as the root of the N-ary tree.
- Left Pointer (First Child): The left child of a node in the Binary Tree points to the first (leftmost) child of that node in the N-ary tree.
- Right Pointer (Next Sibling): The right child of a node in the Binary Tree points to its immediate next sibling in the N-ary tree.



Data structures and its applications

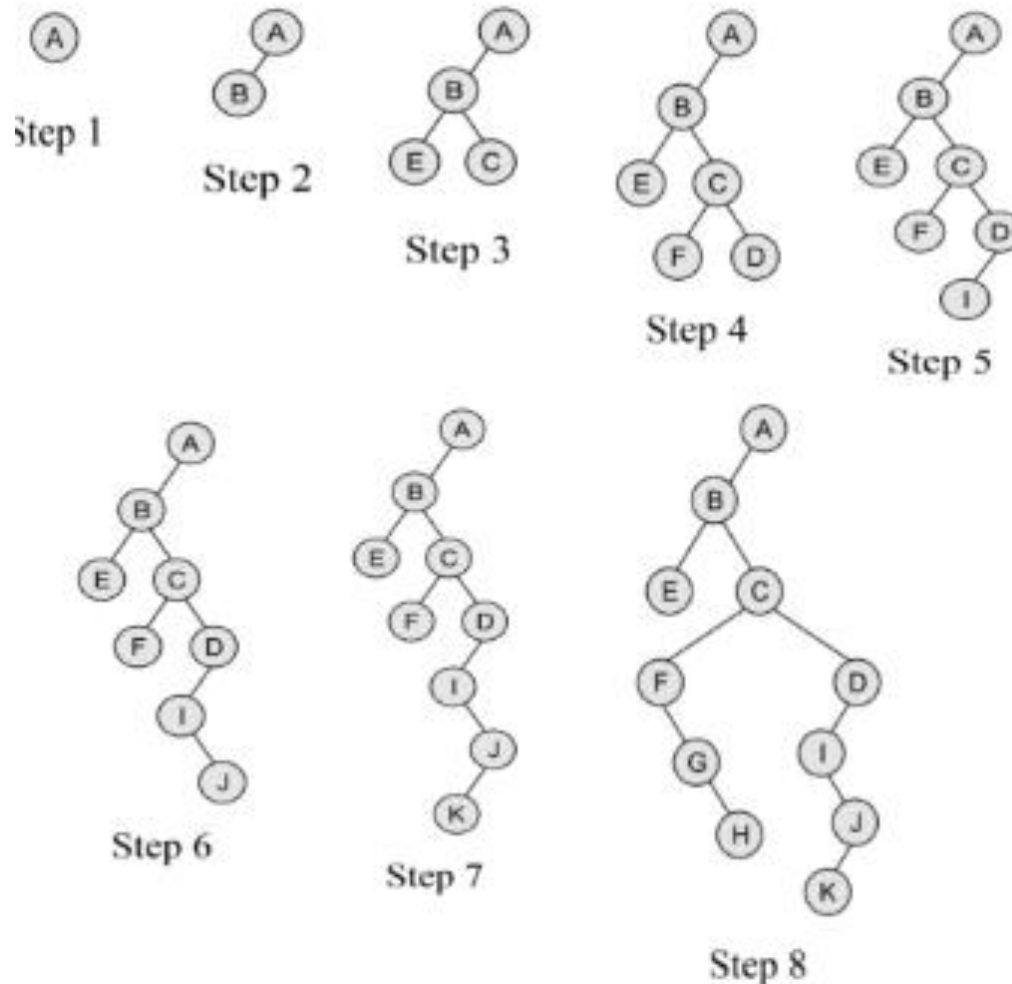
Conversion of an n-ary tree to a binary tree

Convert the following general tree to a binary tree.



Data structures and its applications

Conversion of an n-ary tree to a binary tree



Note: If the parent node has only the right child in the general tree, then it becomes the rightmost child node of the last node following the parent node in the binary tree. In this example, if node B has the right child node L then in binary tree representation L would be the right child of node D.

Data structures and its applications

Conversion of an n-ary tree to a binary tree

The steps:

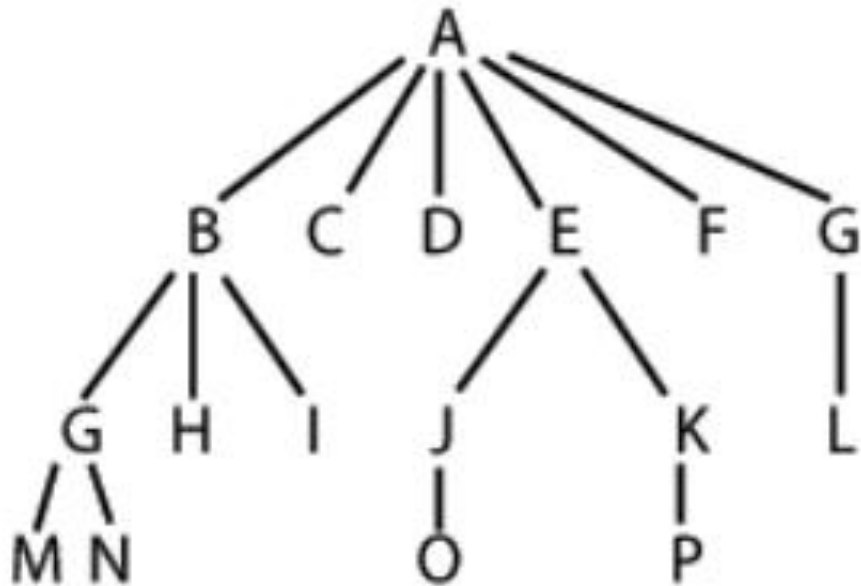
- ✓ As per the rules mentioned above, the root node of general tree A is the root node of the binary tree.
- ✓ The leftmost child node of the root node in the general tree is B and it is the leftmost child node of the binary tree.
- ✓ As B has E as its leftmost child node, it is its leftmost child node in the binary tree whereas it has C as its immediate right sibling node so it is its right child node in the binary tree.
- ✓ C has F as its leftmost child node and D as its immediate right sibling node, so they are its left and right child node in the binary tree respectively.
- ✓ D has I as its leftmost child node which is its left child node in the binary tree but doesn't have any immediate right sibling node, so doesn't have any right child in the binary tree.
- ✓ For I, J is its immediate right sibling node and so it is its right child node in the binary tree.
- ✓ Similarly, for J, K is its leftmost child node and thus it is its left child node in the binary tree.
- ✓ For C, F is its leftmost child node, which has G as its immediate right sibling node, which has H as its just right sibling node and thus they form their left, right, and right child node respectively.



Data structures and its applications

Conversion of an n-ary tree to a binary tree

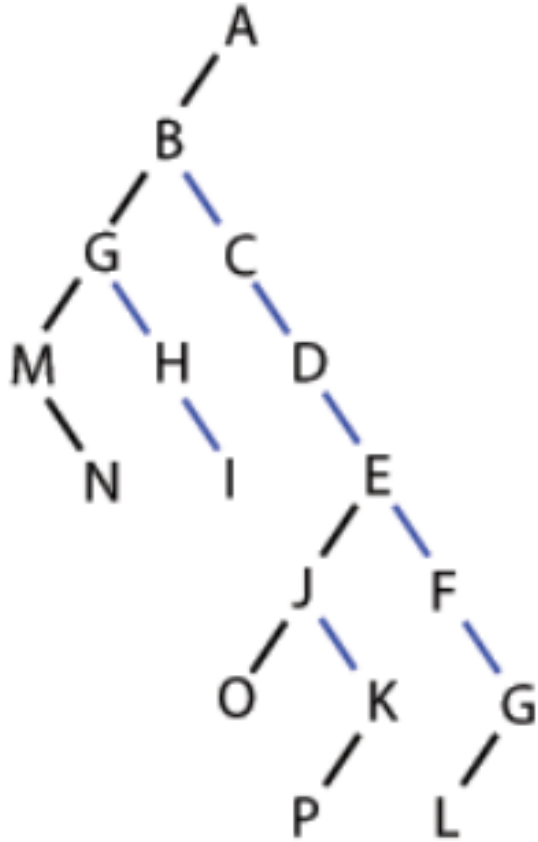
Example2



Data structures and its applications

Conversion of an n-ary tree to a binary tree

Converted binary tree



Data structures and its applications

Binary Search Tree (BST)



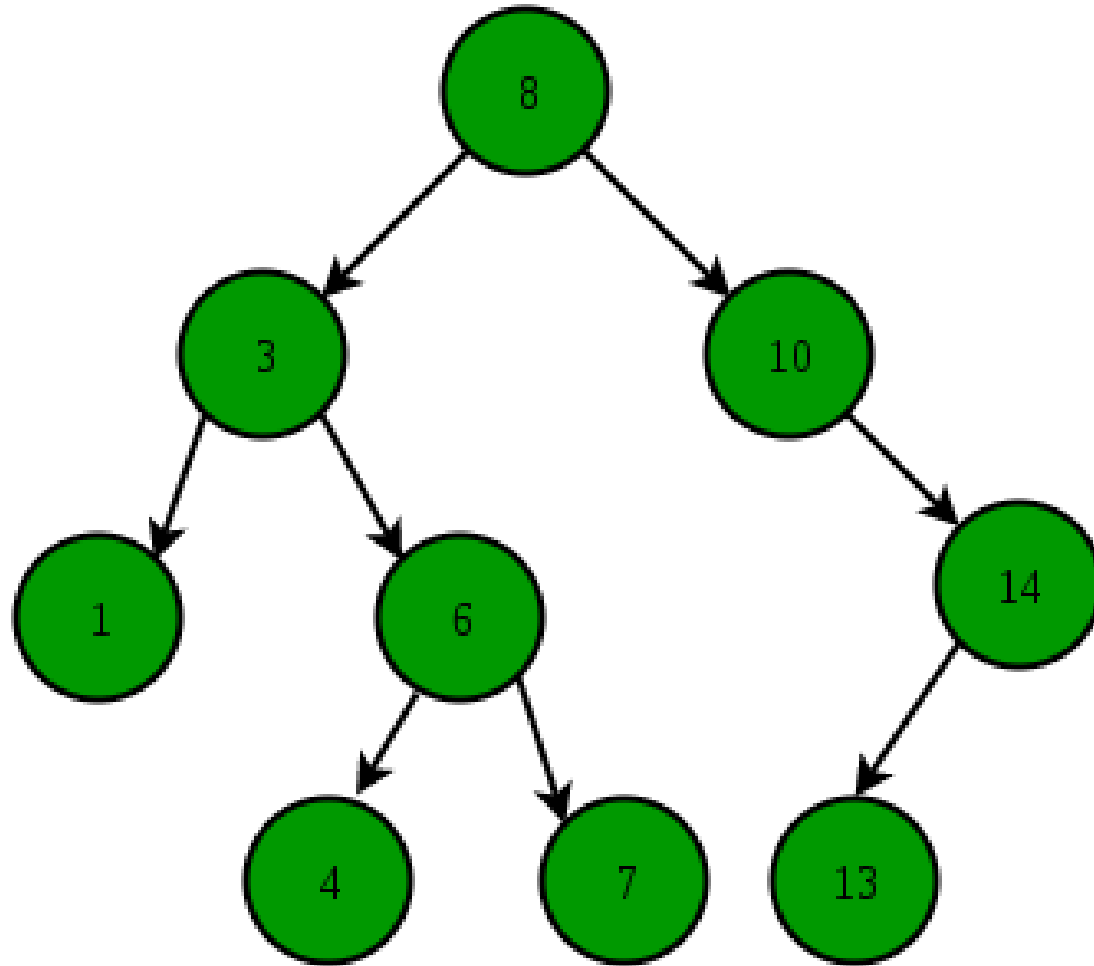
Binary Search Tree is a node-based binary tree data structure which has the following properties:

1. The left subtree of a node contains only nodes with keys lesser than the node's key.
2. The right subtree of a node contains only nodes with keys greater than the node's key.
3. The left and right subtree each must also be a binary search tree. There must be no duplicate nodes.

These properties of BST provide an ordering among keys so that the operations like search, minimum and maximum can be done fast. If there is no ordering, then we may have to compare every key to search a given key.

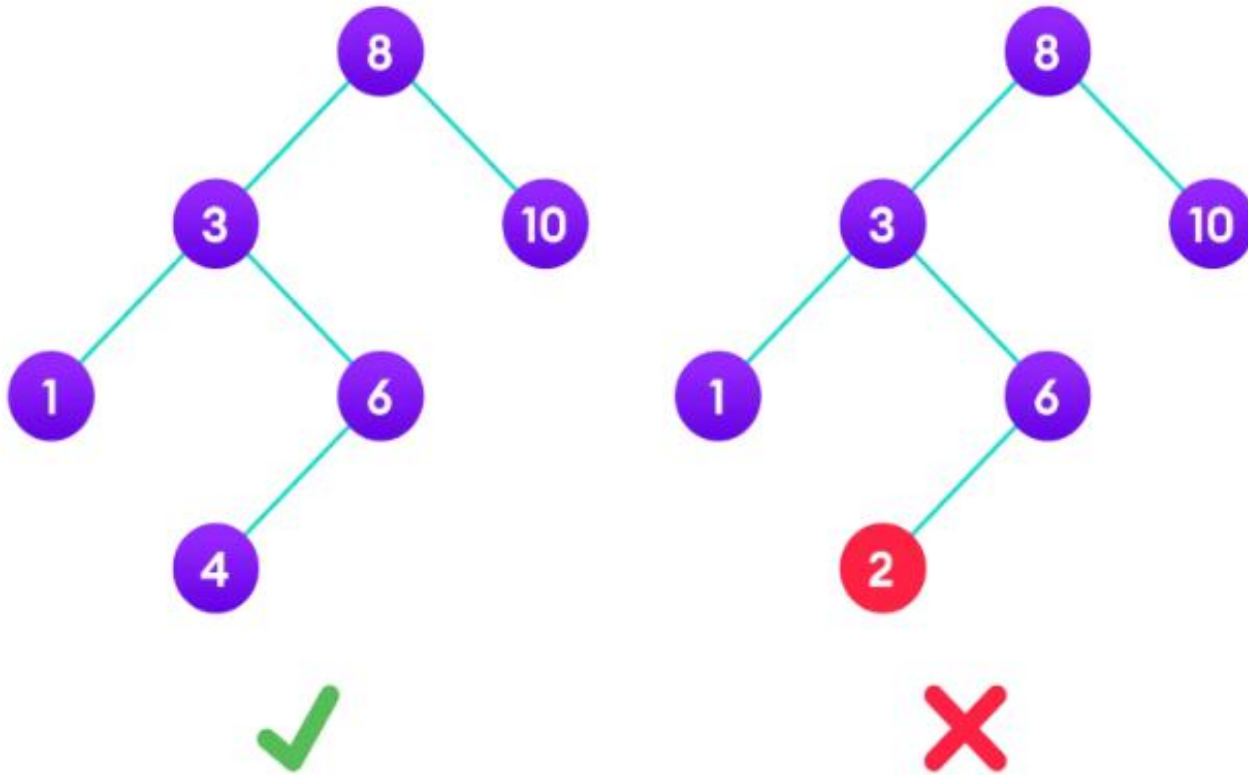
Data structures and its applications

BST – An example



Data structures and its applications

BST – Another example



The binary tree on the right isn't a binary search tree because the right subtree of node 3 contains a value smaller than it.

Data structures and its applications

Operations on BST



Three basic operations can be performed on a binary search tree :

Search Operation

In Search, we have to find a specific element in the data structure. This searching operation becomes simpler in binary search trees because here elements are stored in sorted order.

Algorithm for searching an element in a binary tree is as follows:

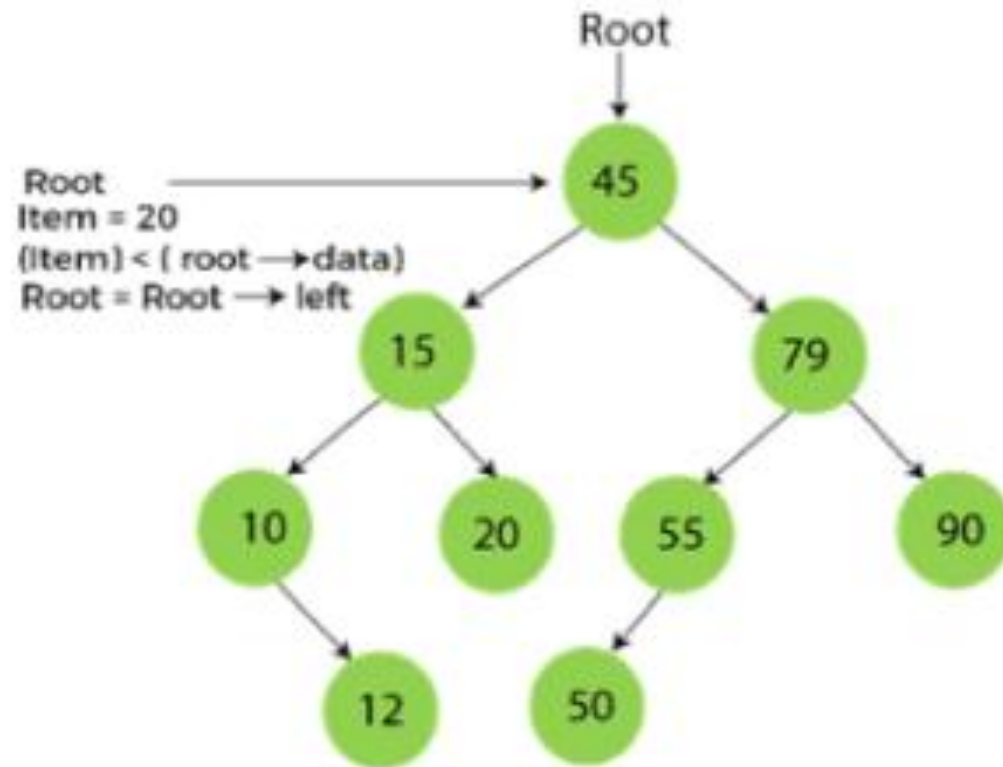
1. Compare the element to be searched with the root node of the tree.
2. If the value of the element to be searched is equal to the value of the root node, return the root node.
3. If the value does not match, check whether the value is less than the root element or not; if it is, then traverse the left subtree.
4. If larger than the root element, traverse the right subtree.
5. **If the element is not found in the whole tree, return NULL.**

Data structures and its applications

Operations on BST

We are searching for item 20.

Step 1

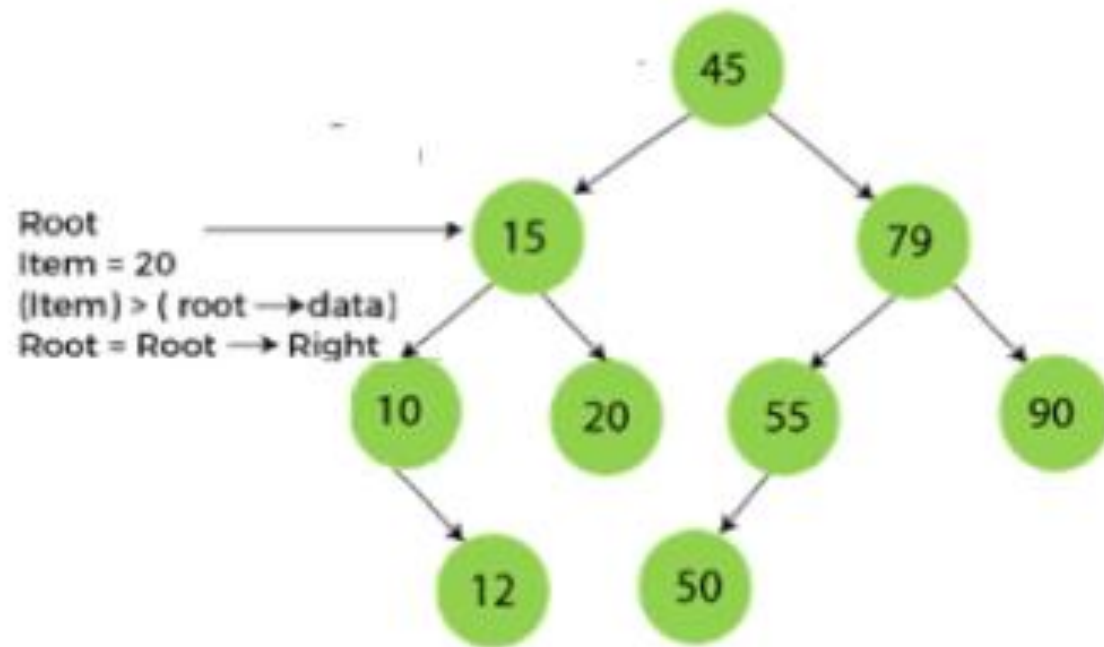


Data structures and its applications

Operations on BST

Step 2

.

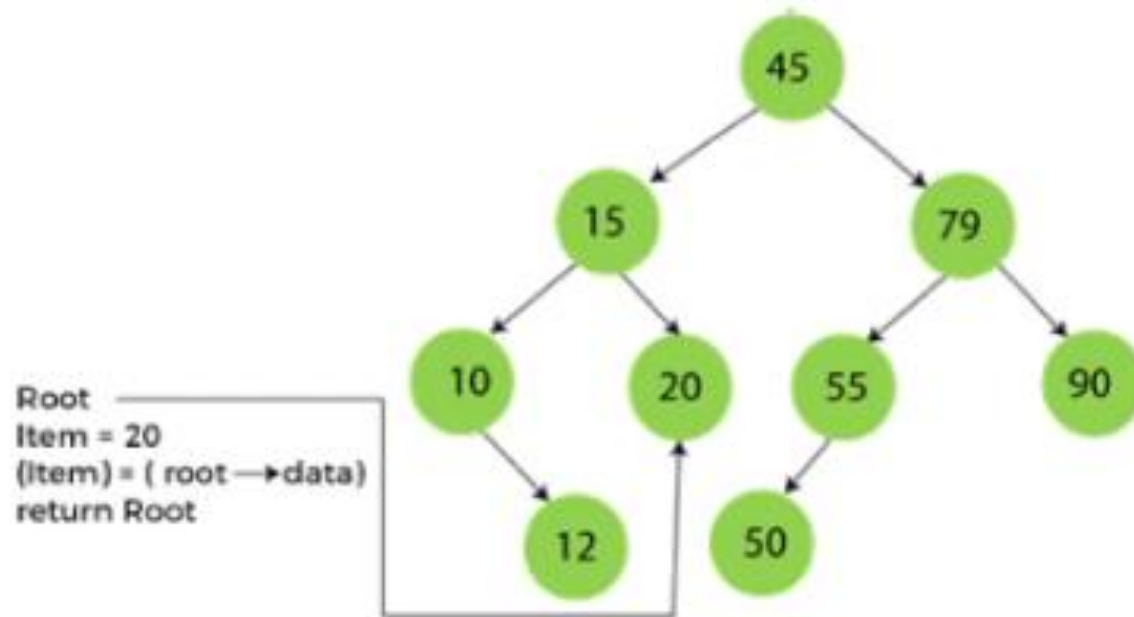


Data structures and its applications

Operations on BST

Step 3

.



Data structures and its applications

Operations on BST ...



Insert Operation

Inserting an element in a binary search tree is always done at the leaf node. To perform insertion in a binary search tree, we start our search operation from the root node, if the element to be inserted is less than the root value or the root node, then we search for an empty location in the left subtree, else, we search for the empty location in the right subtree.

Algorithm for inserting an element in a binary tree is as follows:

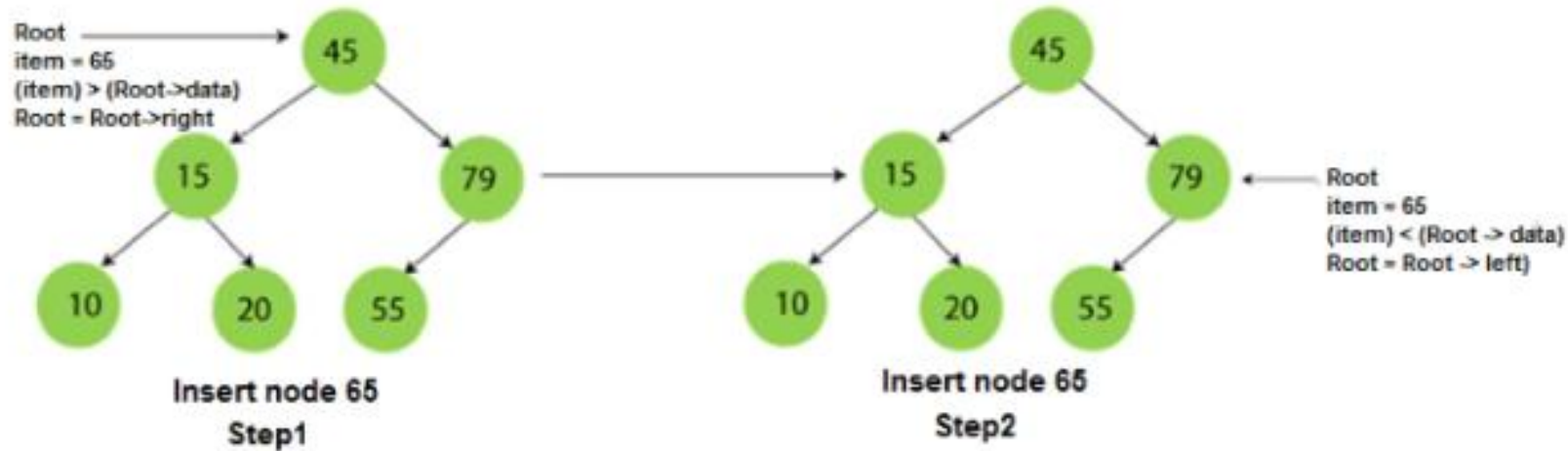
```
if node == NULL
    return createNode(data)
if (data < node->data)
    node->left = insert(node->left, data)
else if (data > node->data)
    node->right = insert(node->right, data)
return node
```

Data structures and its applications

Operations on BST ...

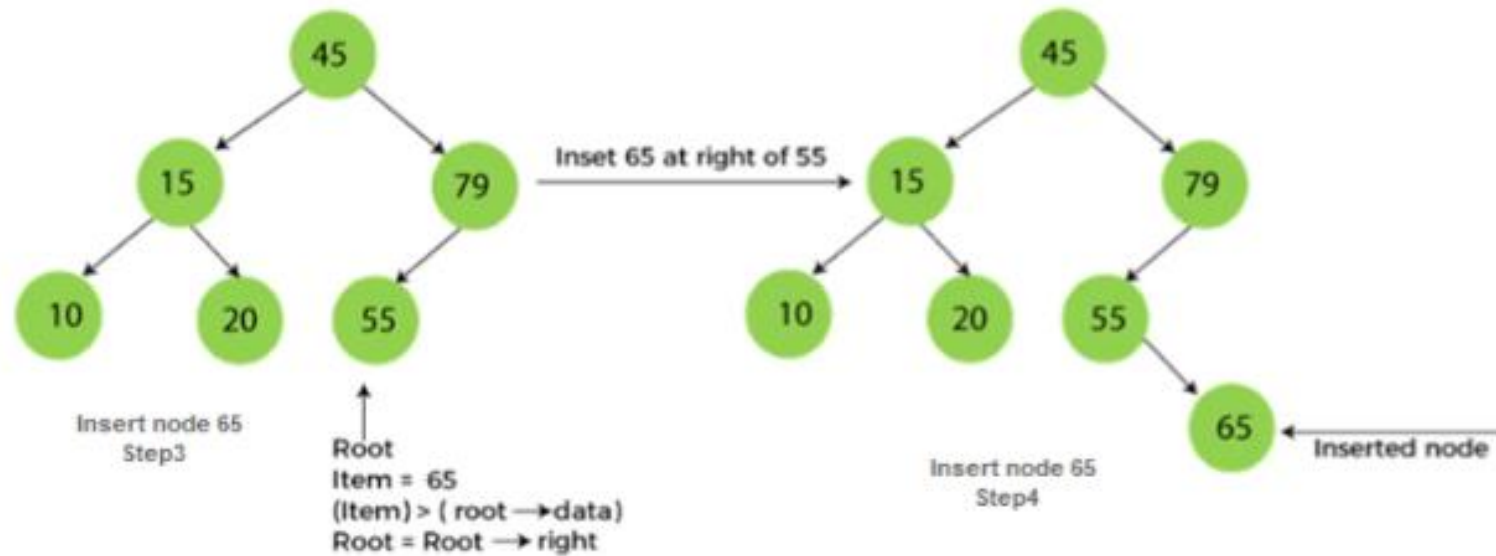
Insert Operation

we are trying to insert an item having a value of 65:



Data structures and its applications

Operations on BST ...



Data structures and its applications

Operations in BST ...



Deletion Operation

In the deletion operation, we have to delete a node from the binary search tree in a way that does not violate its properties. Deletion can occur in three possible cases:

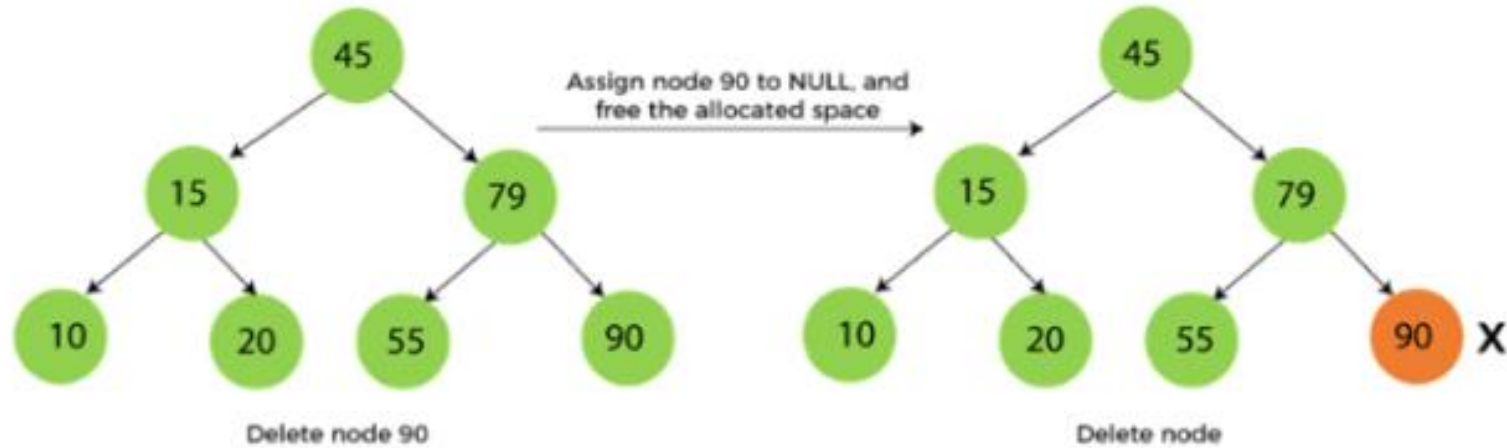
Node to be deleted is the leaf node

This is the simplest case of deleting a node in a binary search tree. Here, we will replace the leaf node with NULL and free the allocated space.

We can understand the deletion operation in a binary search tree with the help of an example where we are trying to delete the node having a value of 90

Data structures and its applications

Operations in BST ...



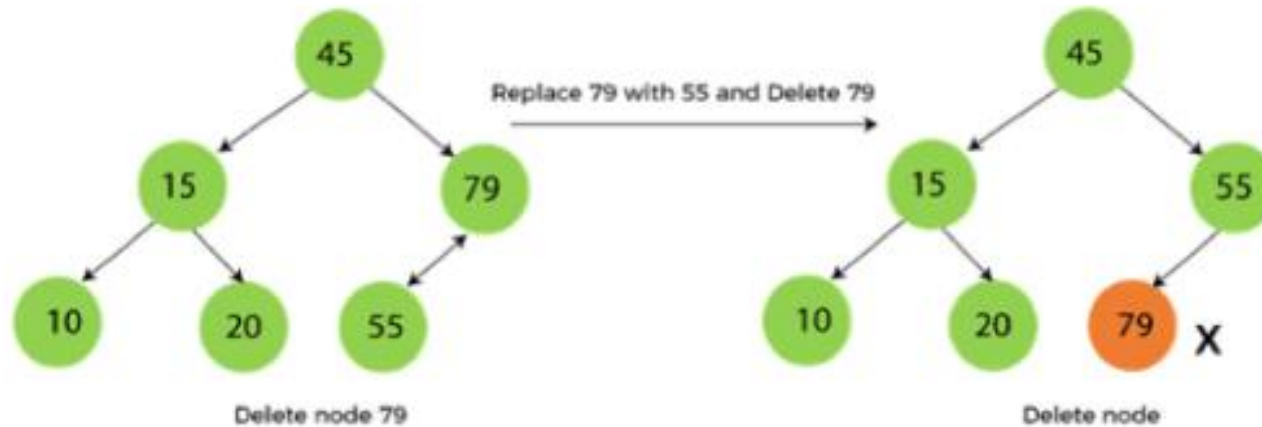
Data structures and its applications

Operations in BST ...

Node to be deleted has a single child node

In this case, we will replace the target node with its child and then delete that replaced child node. This means that the child node will now contain the value to be deleted. So, we will just replace the child node with NULL and free up the allocated space.

In the below example, we have to delete a node having a value of 79, and this node to be deleted has only one child, so it will be replaced with its child 55.



Data structures and its applications

Operations in BST ...



Node to be deleted has both the sub trees

The general algorithm is: to delete N, if it has two subtrees, replace the value in N with the largest value in its left subtree and then delete the node with the largest value from its left subtree.

Note: The largest value in the left subtree will never have two subtrees.

Why?

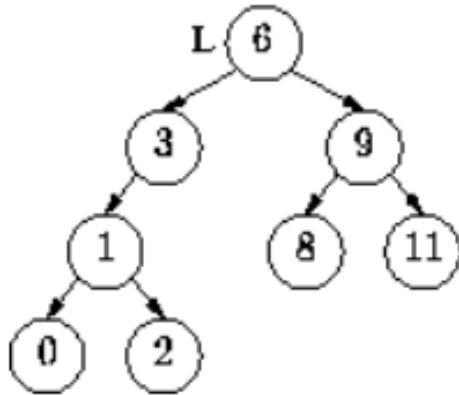
Because if it's the largest value, it cannot have a right subtree.

Can we achieve the same result using the right subtree?

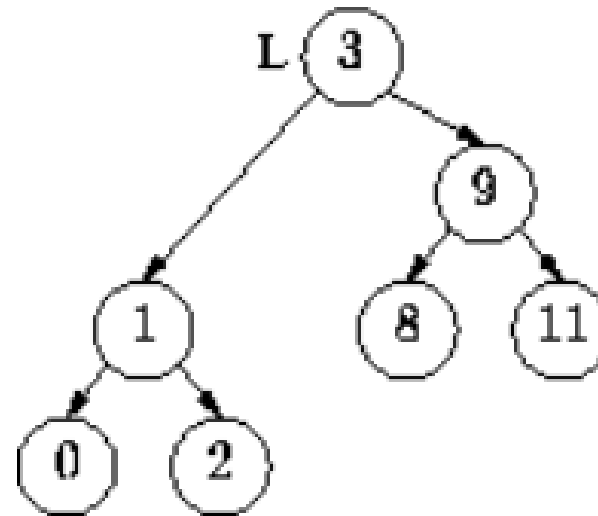
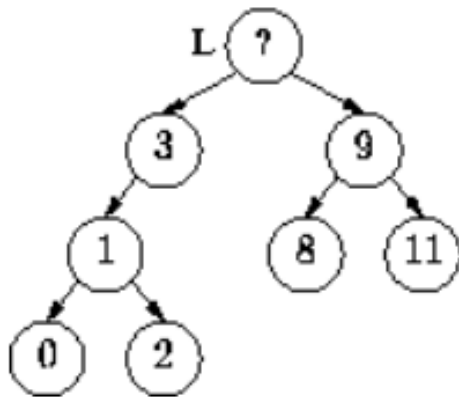
There is nothing special about the left subtree. We could do the same thing with the right subtree: just use the smallest value in the right subtree.

Data structures and its applications

Program to implement BST ...

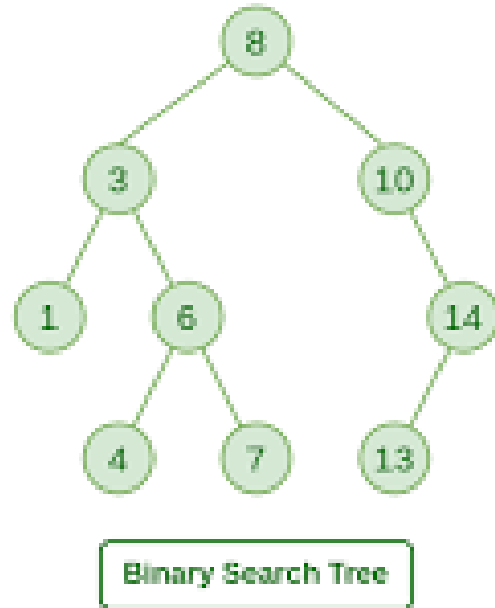


Erase 6, but keep its node:



Data structures and its applications

Program to implement BST ...



Sample implementation is:

[BST_Implement.c](#)

[BST_Implement.h](#)

[BST_main.c](#)

Data structures and its applications

Explanation of the program



The search function checks `if(root==NULL || root->data==x)` which means if the root is NULL then the element is not in the tree and if the root is equal to the target value `x` then the element is found. Otherwise, we implement `else if(x > root->data)`, i.e. if the target element is greater than the root node data, we will search in the right subtree using `search(root->right_child, x)`. Else, in the left subtree using `search(root->left_child, x)`.

In the insert function, if we have to insert a new node to the right child of a node `X`, we will first create a new node that is returned by `insert(root->right_child, x)` and then make the right child of `X` equal to that `root->right_child = insert(root->right_child, x)`. Thus, after searching if we reach a null node, then we just insert a new node there using `if(root==NULL) → return new_node(x)`.

Data structures and its applications

Explanation of the program ...



In the delete function, if the node to be deleted has no children we do `if(root->left_child==NULL && root->right_child==NULL)` which will simply delete the node and free up the allocated space.

While, if the node has only one child, we check `if(root->left_child==NULL)` which means that only the right child exists and we have to delete this node and have to point its parent to its child, so we are storing its child into a temporary variable using `temp = root->right_child` and then deleting the node.

If the node has two children, we find the minimum element of the right subtree using `find_minimum(root->right_child)`. Now, we replace the data of the node to be deleted with the data of this node using `root->data = temp->data`. And then delete the node found by the minimum function using `delete(root->right_child, temp->data)`.



THANK YOU

M S Anand

Department of Computer Science Engineering

anandms@pes.edu